

*Faculty / University banner placeholder
(Drop faculty-logo.png into the project root or screenshots/
folder to replace this frame.)*

University Name

Faculty of Computers and Information

Mohamed Mahmoud Salem
Mohamed Abdellatief
Mariam Mohamed Ahmed
Eman Ahmed Elboghday
Shahd Abdelaziz
Khaled Mohamed

(MSCMS)

A Cloud-Native Microservices Platform
for Modern Sports Club Operations

Team Members

Mariam Mohamed Ahmed
Mohamed Abdellatief
Eman Ahmed Elboghday
Khaled Mohamed
Shahd Abdelaziz
Mohamed Salem

Supervised by: Dr. Shaimaa Saber

May 29, 2026

Contents

1	Introduction	3
1.1	Context	3
1.2	Motivation	3
1.3	Scope	3
1.4	Problem Definition	4
1.5	Goals	5
1.6	Solutions	5
1.6.1	Traditional Solutions	5
1.6.2	Technological Solution	6
2	System Requirements Specification	7
2.1	Functional Requirements	7
2.2	Non-Functional Requirements	10
3	System Models	12
3.1	System Architecture	12
3.1.1	Overview	12
3.1.2	Architectural Decisions	12
3.1.3	Request Lifecycle	13
3.2	Microservices Decomposition	14
3.2.1	Java microservices	14
3.2.2	Machine-learning services	15
3.2.3	Inter-service communication	15
3.3	Use Case Diagram	15
3.3.1	Actors and Use Cases	15
3.3.2	Workflow: Player Registration	15
3.3.3	Workflow: AI Match Prediction	15
3.3.4	Entity Life Cycles	16
3.4	Database Design	16
3.4.1	Polyglot persistence strategy	16
3.4.2	Cross-service identifiers	17
3.4.3	Entity-relationship snapshot	17
3.5	Authentication Flow	17
3.5.1	Identity provider	17
3.5.2	Login flow	18
3.5.3	Refresh-token flow	19
3.5.4	Role-based access control	19
3.5.5	Read-only mode for unprivileged roles	20

3.6	Deployment View	20
3.6.1	Container Topology	20
3.6.2	Build vs. Pull Strategy	20
3.6.3	Volumes and Persistence	21
3.7	System Flowcharts	21
3.7.1	Admin Flow: User Provisioning	21
3.7.2	Head Coach Flow: Plan a Training Session	21
3.7.3	Sponsor Flow: Submit & Track an Offer	22
3.8	Class Diagrams	22
3.8.1	User & Identity	22
3.8.2	Squad & Match Operations	22
3.8.3	Medical Domain	23
3.9	Detailed Entity-Relationship Diagram	23
4	Sequence Diagrams	29
4.1	Diagram Notation	29
4.2	Per-Role Sequence Diagrams	30
4.3	Authentication & Session Sequences	37
4.4	Operational Flow Extensions	38
5	Implementation and Results	41
5.1	Onboarding Page	41
5.2	Login Page	42
5.3	Forgot Password / Refresh-Token Flow	42
5.4	Admin Role Implementation	43
5.5	Sport Manager Role Implementation	46
5.6	Team Manager Role Implementation	49
5.7	Head Coach & Assistant Coaches Role Implementation	52
5.8	Medical Staff Role Implementation	54
5.9	Scout Role Implementation	57
5.10	Sponsor Role Implementation	59
5.11	Performance Analyst Role Implementation	60
5.12	Player Role Implementation	64
5.13	Fan Role Implementation	65
5.14	Notifications (All Roles)	66
5.15	Settings (All Roles)	66
5.16	Cross-cutting Implementation Notes	67
6	Conclusion and Future Work	68
6.1	Comparison Between Sports Management Platforms	68
6.2	Conclusion	69
6.3	What was delivered	69
6.4	Lessons learned	70
6.5	Future work	70

Chapter 1

Introduction

1.1 Context

Modern sports clubs — particularly multi-discipline organisations that operate football, basketball, handball and volleyball teams under one governing body — generate enormous volumes of operational data: training sessions, match events, medical incidents, contract renewals, scouting reports, sponsorship deals and fan engagement. In most clubs this data lives in disconnected spreadsheets, third-party platforms and printed forms, fragmenting the workflow of coaches, doctors, scouts and managers.

1.2 Motivation

The motivation behind MSCMS is to consolidate every operational concern of a multi-sport club into one cohesive platform, where each professional role sees only the information relevant to their job and every data point is traceable, secured, and analytically rich. The design takes deliberate inspiration from FC Barcelona’s real organisational structure: a single governing body (the club) managing four professional teams across football, basketball, handball and volleyball, with shared medical, scouting and sponsorship arms.

1.3 Scope

The system covers eight primary domains:

1. **User & Role Management** — 19 roles across four permission tiers, authenticated via Keycloak.
2. **Player & Team Operations** — rosters, contracts, transfers (incoming, outgoing, call-ups), outer scouting prospects.
3. **Training** — sessions, plans, drills, attendance, and post-session player assessments.
4. **Match Operations** — fixtures, formations, line-ups, in-game events, post-match performance reviews, and player stats.
5. **Medical & Fitness** — injuries, diagnoses, treatments, rehabilitations, recovery programs, fitness tests, training loads.

6. **Reports & Analytics** — match analyses, team-level and player-level analytics, training analytics, scout reports, sponsor contract offers.
7. **Notifications & Alerts** — targeted, role-aware, priority-tagged events delivered in-app or by email.
8. **Machine Learning** — match outcome prediction and player rating estimation served by dedicated FastAPI services.

1.4 Problem Definition

A modern multi-sport club struggles with a constellation of operational problems that no single off-the-shelf tool addresses end-to-end:

- **Fragmented data across departments:** Medical staff, coaching staff, scouting, finance and sponsorship each maintain isolated spreadsheets, PDFs and SaaS tools. A player's injury record is invisible to the coach planning training, and a scout's report is invisible to the sport manager negotiating a transfer.
- **Multi-discipline complexity:** A single club may field football, basketball, handball and volleyball squads under one governing body, each with distinct positions, formations, seasons and KPIs that generic team-management tools cannot model simultaneously.
- **Inconsistent role-based access:** Tools that lack fine-grained RBAC either over-share data (a fan portal that exposes salary information) or over-restrict it (a doctor unable to read training-load data needed to plan rehabilitation).
- **Limited analytical insight:** Without a unified data layer, computing KPIs such as *win-rate per training-load bracket*, *injury rate per playing position*, or *transfer ROI per scout* is impractical.
- **Absent ML predictions:** Few platforms expose match-outcome prediction or quantitative player rating models in a way that non-technical staff can consume directly.
- **Weak sponsor and transfer workflows:** Sponsor offers, contract negotiations and transfer windows are typically managed over email and PDF, with no audit trail or notification fan-out.
- **Poor fan engagement:** Fans cannot follow upcoming fixtures, results, squads or news through the same system the club uses internally, fragmenting communication and brand identity.
- **Manual notifications:** Critical alerts (injury, line-up change, transfer approval, sponsor decision) reach the relevant actor late or not at all because there is no role-aware notification fabric.
- **Brittle deployments:** Monolithic legacy systems force every domain (medical, training, sponsorship) onto the same release cadence, blocking unrelated feature work.
- **Inconsistent identity:** Without a central identity provider, every department maintains its own user accounts, making onboarding, off-boarding and audit slow and error-prone.

1.5 Goals

MSCMS pursues the following measurable, role-aware goals:

- **Unify operations** into a single platform spanning the eight domains in Section 1.4.
- **Enforce 19-role RBAC** consistently at three layers — Keycloak (identity), API gateway (token verification), and Next.js middleware (route guards) — so the same permission rules apply whether a user navigates via UI or calls APIs directly.
- **Decompose the back-end** into seven independently deployable Spring Boot microservices, discoverable through Eureka and reachable through a single API gateway.
- **Model real multi-sport structure** by seeding football, basketball, handball and volleyball squads, each with their own positions, contracts, fixtures and rosters, taking FC Barcelona as the working example.
- **Centralise analytics** into role-aware dashboards that aggregate team-, player-, match-, training- and medical-level data into a single overview surface.
- **Integrate machine learning** by serving match-outcome prediction and player-rating models from dedicated FastAPI services consumed transparently through the Next.js front-end.
- **Streamline sponsor and transfer workflows** by digitising sponsor offers, transfer windows, call-ups and contract approvals — each producing notifications, audit events and analytics entries.
- **Deliver a polished fan experience** with read-only access to fixtures, results, squads, news and team data, themed to the modelled club’s branding (Barcelona blue and red, Barlow Condensed typography).
- **Provide reliable notifications** through a dedicated notification-mail-service that fans out role-aware alerts in real time and as digestible in-app feeds.
- **Guarantee independent deployability** so each domain ships on its own release cadence behind backward-compatible API contracts.

1.6 Solutions

1.6.1 Traditional Solutions

Historically, multi-sport clubs have relied on a combination of manual and semi-digital tools to coordinate their operations. Coaching staff typically maintain training plans and line-ups in spreadsheets or printed binders, medical departments record injuries and treatments in paper charts or local Excel sheets, and scouts compile player reports as Word or PDF documents that circulate by email. Sponsorship and transfer negotiations are conducted over email threads with attached contracts, while fan communication is handled separately through generic websites and social-media channels. This patchwork places a heavy coordination burden on administrators: data is duplicated, reconciliations are manual, and any change to a player’s status (an injury, a transfer, a contract renewal)

requires multiple parallel updates across disconnected silos. The approach scales poorly with the number of disciplines (football, basketball, handball, volleyball) and makes cross-domain analytics — such as correlating training load with injury frequency or transfer ROI with scout reports — effectively impossible. Furthermore, traditional approaches lack consistent role-based access control: sensitive medical or financial data is either over-shared via shared drives or over-restricted to the point that legitimate staff cannot do their job.

1.6.2 Technological Solution

To overcome these limitations, we developed **MSCMS**, a cloud-native multi-sport club management platform built on a modern microservice architecture. The system unifies every operational concern of a multi-discipline club — football, basketball, handball, volleyball — into a single coherent interface, while preserving independent deployability of each functional domain. The back-end consists of seven Spring Boot microservices (user, player, training-match, medical-fitness, notification-mail, reports-analytics, plus the gateway/discovery/config infrastructure) and two FastAPI machine learning services (match outcome prediction and player rating estimation). All services are registered with a Eureka discovery server, configured through Spring Cloud Config, and reached through a single Spring Cloud Gateway that performs JWT verification, request routing and CORS handling.

Authentication is delegated to Keycloak (realm: `mscms`) which manages 19 distinct roles spanning club leadership (Admin, Sport Manager, Team Manager), coaching staff (Head Coach, Assistant Coach, Goalkeeper Coach), medical and fitness staff (Doctor, Physiotherapist, Fitness Coach), analytics roles (Performance Analyst, Scout), commercial roles (Sponsor), and end-users (Player, Fan, National Team). JWTs issued by Keycloak are verified at the gateway and decoded by the Next.js middleware to enforce the same permission map on both layers. Each microservice owns its own PostgreSQL database, and asynchronous events flow through Kafka and RabbitMQ, with Redis providing a shared cache. Two ML services serve pre-trained joblib models behind a Next.js rewrite proxy so the front-end can call them through the same origin. The result is a scalable, observable, role-aware platform that addresses the fragmentation, complexity and analytical blindness of traditional approaches in one cohesive system.

Chapter 2

System Requirements Specification

2.1 Functional Requirements

MSCMS supports nineteen roles grouped into four permission tiers and nine functional clusters. Each role sees a tailored navigation, dashboard and CRUD surface enforced uniformly at the gateway and the front-end middleware.

Onboarding and Authentication

- **Onboarding Page:** Displays a landing page with the club identity, a brief description of MSCMS, and a *Login* button.
- **Login:** Validates credentials against Keycloak using the OAuth 2.0 password grant. On success, the access token is decoded client-side; `sub` is stored as `keycloakId` and the user's role is stored in `localStorage` and as a cookie consumed by the Next.js middleware.
- **Refresh Token:** Transparent re-authentication via the refresh-token grant; on failure, the front-end clears storage and redirects to `/login`.
- **Role-Based Redirect:** After login, users land on a single `/dashboard` page whose contents are computed from their role.

Admin Role

- **User Management:** Create, edit, deactivate and delete users across all 19 roles via the user-management-service backed by Keycloak Admin API.
- **Staff Management:** Onboard sport managers, team managers, coaching staff, doctors, physios, scouts and sponsors with role assignment.
- **System-wide Dashboard:** Aggregated counts of users, teams, players, matches and outstanding tasks.
- **Notification Broadcasts:** Issue role-targeted alerts and announcements to any subset of users.
- **Audit View:** Read access to every domain (players, teams, training, medical, reports) for governance.

Sport Manager Role

- **Multi-team Oversight:** View and manage every team across football, basketball, handball and volleyball.
- **Transfers:** Approve incoming transfers, outgoing transfers and call-ups; review scout reports.
- **Sponsor Contracts:** Review pending sponsor offers and accept or reject them.
- **Strategic Analytics:** Aggregated team and player performance across disciplines.

Team Manager Role

- **Single-team Operations:** Manage roster, contracts, fixtures, line-ups and reports for one team.
- **Match Operations:** Create matches, attach formations, publish line-ups, log in-game events and request post-match reviews.
- **Training Plans:** Approve or adjust training plans proposed by the head coach.
- **Player Calls:** Issue call-ups to national teams.

Head Coach & Assistant Coaches

- **Training Sessions:** Create training plans, drills and sessions; assign player attendance and assessments.
- **Tactical Formations:** Design and save formations; publish line-ups for upcoming matches.
- **Match Events:** Record live in-game events (goals, cards, substitutions) and post-match reviews.
- **Player Stats:** View and analyse aggregated player performance per match and per training session.

Medical Staff (Doctor, Physio, Fitness Coach)

- **Injury Records:** Log injuries, diagnoses, treatments and rehabilitations per player.
- **Recovery Programs:** Plan and track recovery programs with progress milestones.
- **Fitness Tests & Training Load:** Capture fitness-test results and per-session training-load measurements.
- **Cross-domain Visibility:** Read access to training sessions and match minutes for context-aware rehab planning.

Scout Role

- **Outer Players Catalogue:** Browse external prospects across rival teams and external markets.
- **Scout Reports:** Create, edit and delete scout reports with structured ratings and free-text observations.
- **Recommendation Pipeline:** Forward reports to sport manager for transfer consideration.

Sponsor Role

- **Active Contracts:** View own contract status and history.
- **Submit Offers:** Submit new sponsorship offers with value, duration and conditions.
- **Notifications:** Receive decisions (accepted / rejected / pending) from sport manager.

Performance Analyst Role

- **Match Analyses:** Author detailed analyses linked to matches with metrics, narrative and recommendations.
- **Player Analytics:** Compute and publish per-player analytics with KPIs spanning multiple matches.
- **Team Analytics:** Roll up team-level KPIs over a season.
- **AI Predictions:** Consume the ML services for match outcome prediction and player rating estimation.

Player Role

- **Personal Schedule:** View own training sessions and match fixtures.
- **Performance View:** Read-only access to own match statistics, training assessments and aggregated analytics.
- **Medical View:** Read-only view of own injury status, treatments and recovery progress.
- **Notifications:** Receive line-up announcements, call-ups, and training updates.

Fan Role

- **Match Calendar:** View upcoming and past fixtures across all disciplines.
- **Teams & Players Directory:** Read-only access to team rosters and player profiles.
- **Public Reports:** Read public-facing match analyses and team highlights.
- **Training Highlights:** Read-only view of public-facing training stories.
- **Restricted Surface:** Add / edit / delete buttons are hidden; medical, scouting, sponsor and finance views are invisible to fans.

National Team Role

- **Call-up Workflow:** Receive and accept call-ups for players from the club.
- **Outgoing Player View:** Read-only access to the scheduled training and match availability of called-up players.

2.2 Non-Functional Requirements

Technology Stack

- **Front-end:** Next.js 16 (App Router) + React 19, Tailwind CSS v4, Barlow Condensed typography, react-icons, chart.js / Recharts, jwt-decode, js-cookie.
- **Back-end:** Java 17 + Spring Boot 3.x, Spring Cloud Config / Eureka / Gateway, Spring Data JPA + Hibernate.
- **Identity:** Keycloak 26.2.5 with the `mscms` realm and 19 realm roles.
- **Persistence:** PostgreSQL 14 (database-per-service: `mscms_user`, `mscms_player`, `mscms_training`, `mscms_medical`, `mscms_notification`, `mscms_reports`).
- **Messaging:** Apache Kafka 7.5.0 for high-volume event streams, RabbitMQ 3 for transactional notifications.
- **Cache:** Redis 7-alpine.
- **ML services:** Python 3 + FastAPI + Uvicorn, joblib model serialisation (`barca_prediction_model`, `player_evaluation_model.joblib`).
- **Containerisation:** Docker Engine + Docker Compose (`docker-compose.full.yml`).

Security

- JWT-based stateless authentication issued by Keycloak.
- Tokens verified at the gateway against Keycloak's JWKS endpoint, with realm-role claims propagated to downstream services via headers.
- Role-based access control enforced *twice*: once at the gateway and once at the Next.js middleware (defence in depth).
- HTTPS terminations expected at the reverse proxy / load balancer in production deployments.
- Username / password regexes prevent invalid characters at creation time (matching Keycloak validation rules).

Performance

- Independent scaling per microservice — the gateway and notification-mail-service can be horizontally scaled independently of the medical-fitness-service.
- Redis-backed caching for hot lookups (user profile by `keycloakId`).
- ML inference served behind a same-origin Next.js rewrite to avoid CORS pre-flights and to allow caching at the edge if desired.
- Idempotent seeders (`count() > 0` guard) so a redeploy does not duplicate data.

Scalability

- Horizontally scalable Spring Boot services discoverable through Eureka.
- Database-per-service prevents cross-domain lock contention and allows per-domain database scaling strategies (read replicas, partitioning).
- Event-driven fan-out via Kafka / RabbitMQ for high-volume notification streams.

Reliability

- Healthcheck endpoints on every service consumed by Docker Compose and ready to be wired into Kubernetes liveness / readiness probes.
- Friendly toast notifications surface API errors to the user without exposing stack traces.
- Idempotent seeders ensure consistent baseline state across redeployments.

Maintainability

- Single source of truth files for cross-cutting concerns: `permissions.js` on the front-end, `SeedIds.java` on the back-end.
- Each microservice has its own Maven module, Dockerfile and bootstrap seeder, allowing independent build and release.
- Spring Cloud Config externalises configuration so per-environment overrides do not require code changes.
- OpenAPI / Swagger UI shipped per service for live API exploration.

Observability

- Each Spring Boot service exposes `/actuator/health`, `/actuator/info` and `/actuator/metrics`.
- Eureka dashboard for service-registration visibility.
- Centralised application logs ready for Loki / ELK integration.

Chapter 3

System Models

This chapter groups the four design views of MSCMS: *architecture*, *microservices decomposition*, *database design*, *authentication flow*, *deployment topology* and the *use case diagram*. Each is expressed as a TikZ figure plus a short explanation. Per-operation sequence diagrams have their own chapter (Chapter 4).

3.1 System Architecture

3.1.1 Overview

MSCMS follows the classic *API Gateway + Service Discovery + Centralised Configuration* pattern recommended by Spring Cloud documentation. Figure 3.1 shows the high-level view: every external request enters through the Gateway, which delegates to the right internal microservice after JWT validation. Each microservice owns its own PostgreSQL database, and cross-service communication is either synchronous (internal REST through the gateway) or asynchronous (Kafka events).

3.1.2 Architectural Decisions

Why microservices?

Each operational domain (medical, training, reports) has its own data ownership, release cadence and access patterns. Decomposing along these seams lets us:

- scale the high-traffic services (player, training-match) independently;
- evolve the schema of one domain without coordinating migrations across the whole codebase;
- rebuild and redeploy one service’s container without touching the others.

Why Eureka + Config Server?

The dynamic IPs assigned to Docker containers make hard-coded service URLs brittle. Eureka acts as the source of truth for *which container holds which service right now*, and the Spring Cloud Config Server centralises properties (database URLs, JWT issuer) so they can change without rebuilding images.

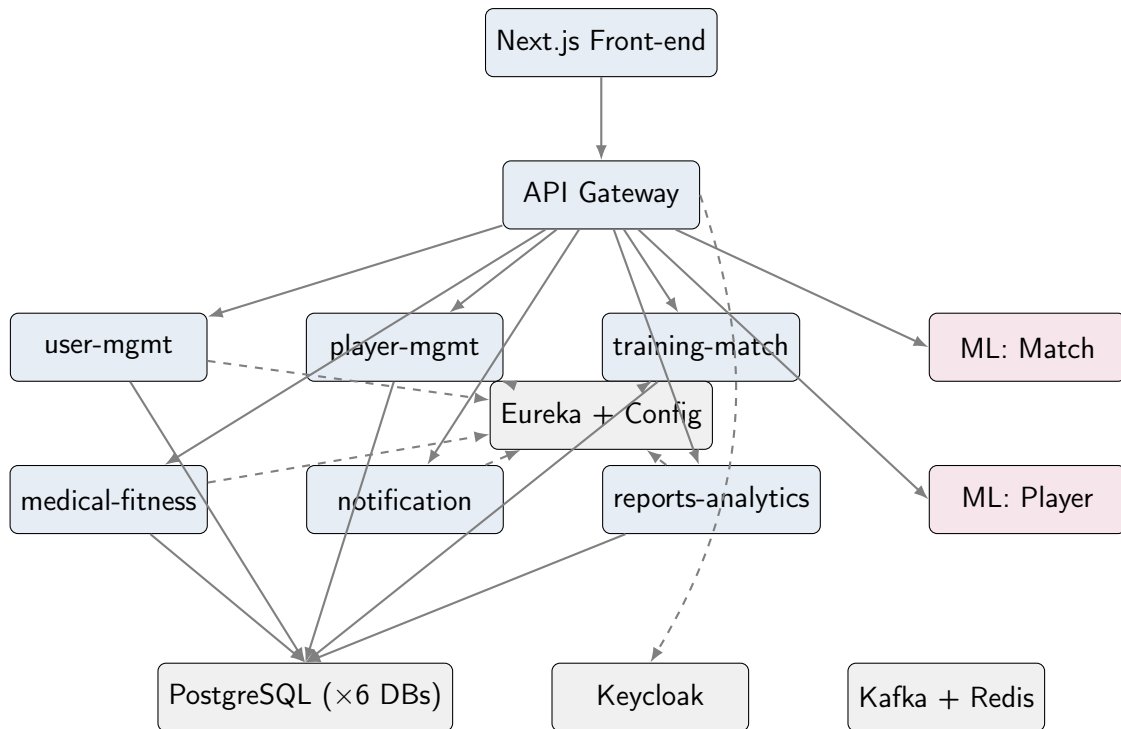


Figure 3.1: High-level MSCMS architecture. Solid arrows are synchronous HTTP; dashed arrows are control-plane (service registration, JWT validation).

Why a separate Gateway?

The gateway is the only externally-exposed Spring service. It validates JWT tokens, applies global filters (CORS, logging), and routes by URI prefix (`/players/**` → `player-management-service`). This keeps each business service oblivious to authentication concerns and produces a single public surface for the front-end to call.

Why a Python ML tier?

The match prediction and player rating models are pre-trained `scikit-learn` pipelines serialised with `joblib`. Serving them from Java would require a brittle bridge; instead, the same Docker image hosts two FastAPI applications, one per model, communicating with the front-end over a Next.js rewrite proxy to side-step CORS at the browser boundary.

3.1.3 Request Lifecycle

A typical authenticated request follows this path:

1. Front-end attaches the JWT from `localStorage` as an `Authorization: Bearer` header and posts to the gateway.
2. Gateway validates the JWT signature against Keycloak's JWKS, extracts the role claim, and routes by URI prefix.
3. The downstream microservice executes the business logic, reads/writes its private PostgreSQL database, and returns the response.

4. For asynchronous events (e.g. "injury reported"), the service publishes a Kafka topic that the notification service consumes to dispatch alerts.

3.2 Microservices Decomposition

The system contains seven **Java/Spring Boot** services and two **Python/FastAPI** services. The infrastructure containers (PostgreSQL, Redis, Kafka, RabbitMQ, Keycloak) are managed by the same Compose stack but considered platform components rather than first-class business microservices.

3.2.1 Java microservices

Service	DB	Responsibility
eureka-server	—	Service registry. Every business service registers here on boot.
config-server	—	Centralised <code>application.yml</code> distribution. Reads from a Git repo or class-path.
gateway-service	—	Spring Cloud Gateway. Validates JWT, routes by URI prefix, applies CORS and logging filters. Public entry point on port 8080 .
user-management-service	mscms_user	Owns the user-profile data for every role (19 roles). Talks to Keycloak Admin API to provision new users. Provides <code>/auth/login</code> , <code>/auth/signup</code> and <code>/auth/admin/create-user</code> . Also owns <code>NationalTeam</code> records.
player-management-service	mscms_player	Sports, teams, rosters, contracts, outer (scouting) players, outer teams, incoming/outgoing transfers, and national-team call-ups.
training-match-service	mscms_training	Training sessions, plans, drills, attendance, player training assessments. Match fixtures, formations, line-ups, in-game events, performance reviews, and per-player match statistics.
medical-fitness-service	mscms_medical	Injuries, diagnoses, treatments, rehabilitation programs, recovery programs, fitness tests, and training loads.
notification-mail-service	mscms_notification	In-app notifications, role-targeted alerts, and outbound e-mail. Subscribes to Kafka topics from the other services.

Service	DB	Responsibility
reports-analytics-services	msms_reports	Cross-domain analytics: match analyses, player analytics, team analytics, training analytics, scout reports, and sponsor contract offers.

3.2.2 Machine-learning services

Service	Behaviour
ml-match-predictor (port 9000)	FastAPI exposing POST /predict with {home_team, away_team, referee_name}. Loads FCB_Football_Matches.xlsx and a pre-trained joblib model. Returns {prediction: W D L, probabilities}.
ml-player-rating (port 9001)	FastAPI exposing GET /predict/{player_name}. Loads Football_Players_Data.csv (FC Barcelona squad) and a joblib regression model. Returns predicted rating and the actual dataset rating if available.

3.2.3 Inter-service communication

- **Synchronous** REST through the gateway. The gateway forwards external traffic; internal-to-internal calls go via Eureka-resolved service IDs (e.g. `http://user-management-service/`).
- **Asynchronous** Kafka topics. Producer services (training-match, medical-fitness) publish domain events (e.g. `InjuryReported`, `MatchFinished`); the notification service consumes them to fan out alerts and e-mails.

3.3 Use Case Diagram

3.3.1 Actors and Use Cases

Figure 3.2 summarises the principal actors of MSCMS and the high-level use cases each can perform. Roles are grouped by privilege tier; a single user may carry multiple realm roles, in which case the union of their use cases applies.

3.3.2 Workflow: Player Registration

The most security-sensitive workflow is creating a new player. It crosses the front-end, the gateway, Keycloak and two databases. Figure 3.3 renders the activity diagram.

3.3.3 Workflow: AI Match Prediction

Figure 3.4 captures the simpler ML flow. Note that the front-end calls the FastAPI service through a Next.js *rewrite proxy* to avoid CORS, not through the gateway.

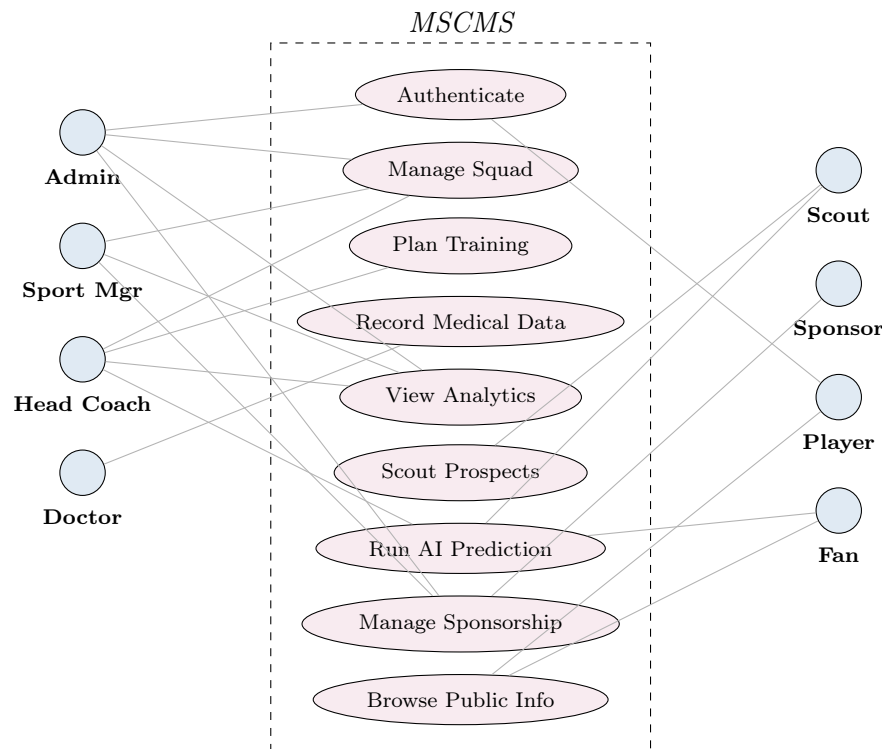


Figure 3.2: Use-case diagram. Each association links an actor to the use cases they can invoke. All actors share the *Authenticate* dependency.

3.3.4 Entity Life Cycles

The system tracks several stateful entities. Figures 3.5, 3.6 and 3.7 render their state machines.

3.4 Database Design

3.4.1 Polyglot persistence strategy

The data layer follows the *database-per-service* pattern. Each Java microservice owns one PostgreSQL database; cross-service references use opaque identifiers (Keycloak UUIDs for users, foreign-key IDs for entities) rather than direct database joins. This isolates each service's schema evolution.

Database	Primary tables
mscms_user	user_profile, player_profile, head_coach_profile, doctor_profile, physiotherapist_profile, fitness_coach_profile, team_manager_profile, sport_manager_profile, scout_profile, sponsor_profile, fan_profile, national_team_profile

<code>mscms_player</code>	<code>sports,</code> <code>player_contracts,</code> <code>outer_players,</code> <code>player_transfer_outgoing,</code> <code>player_call_up_requests</code>	<code>teams,</code> <code>player_transfer_incoming,</code>	<code>rosters,</code> <code>outer_teams,</code>
<code>mscms_training</code>	<code>training_sessions,</code> <code>training_drills,</code> <code>player_training_assessments,</code> <code>match_formation,</code> <code>match_events,</code> <code>player_match_statistics</code>	<code>training_plans,</code> <code>training_attendance,</code> <code>matches,</code> <code>match_lineups,</code> <code>match_performance_reviews,</code>	
<code>mscms_medical</code>	<code>injuries, diagnoses, treatments, rehabilitations,</code> <code>recovery_programs,</code> <code>training_loads</code>	<code>fitness_tests,</code>	
<code>mscms_notification</code>	<code>notifications, alerts, messages</code>		
<code>mscms_reports</code>	<code>match_analysis,</code> <code>team_analytics,</code> <code>scout_reports,</code>	<code>player_analytics,</code> <code>training_analytics,</code> <code>sponsor_contract_offers</code>	

3.4.2 Cross-service identifiers

Three classes of identifier are shared across databases:

- **Keycloak UUID** (`keycloakId`) — the canonical identity for a person. Every service stores this string instead of a numeric FK to `mscms_user`.
- **Numeric domain IDs** — e.g. `teamId`, `playerId`, `matchId`, `sportId`. These are the JPA auto-generated primary keys of `mscms_player` and `mscms_training`. The seeders preserve a deterministic insert order so that `teamId = 1` always means *FC Barcelona* across every service.
- **Realm role names** — string identifiers (e.g. `ADMIN`, `HEAD_COACH`) embedded in the JWT.

3.4.3 Entity-relationship snapshot

Figure 3.8 sketches the principal relationships among the business entities (see the dedicated `ER_DIAGRAM.md` file for the full diagram in Mermaid form).

3.5 Authentication Flow

3.5.1 Identity provider

Keycloak hosts the realm `mscms`, bootstrapped from `keycloak/mscms-realm.json` at first boot. The realm declares 19 roles, an OIDC client `mscms-frontend` (public, standard flow + direct grants enabled), and 16 seeded users covering every role.

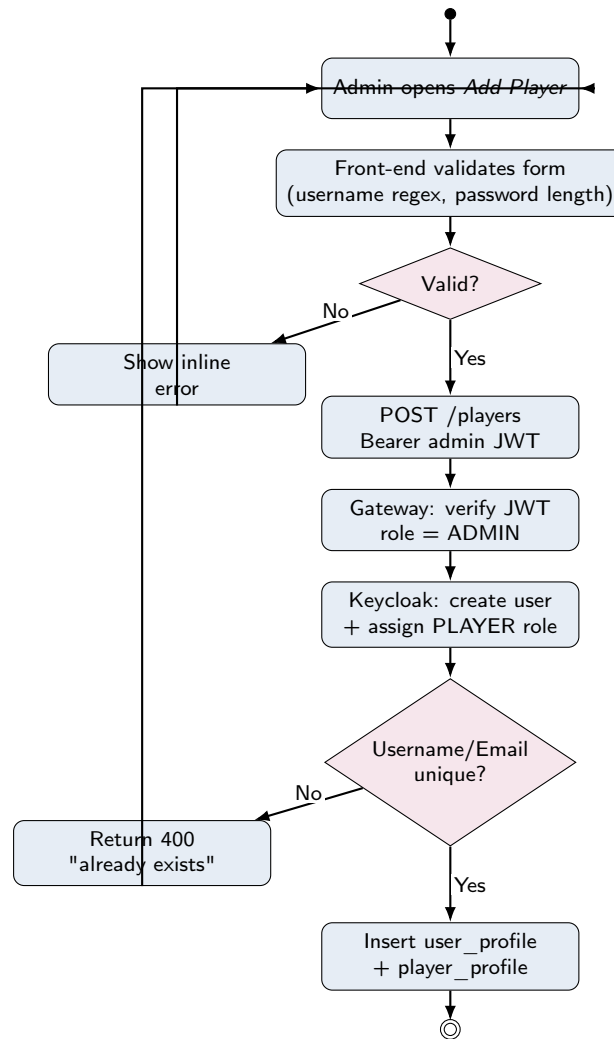


Figure 3.3: Activity flow: registering a new player. The two decision nodes correspond to front-end validation and Keycloak uniqueness checks; either failure routes the user back to the form.

3.5.2 Login flow

The login sequence is shown in Figure 3.9; the textual walk-through is:

1. The user submits `username` + `password` to the front-end's `Login` page.
2. The front-end posts to `POST http://localhost:8080/auth/login` on the gateway.
3. The gateway forwards to `user-management-service`, which calls Keycloak's `token` endpoint with the password grant.
4. Keycloak returns a JWT `access_token` (and a refresh token).
5. The response propagates back to the browser; the front-end stores the access token in `localStorage` and the role (`realm_access.roles[0]`) in both `localStorage` and an `HttpOnly`-style cookie consumed by Next.js middleware.
6. Subsequent requests attach the JWT as `Authorization: Bearer <token>`; the gateway validates the signature against Keycloak's JWKS before forwarding.

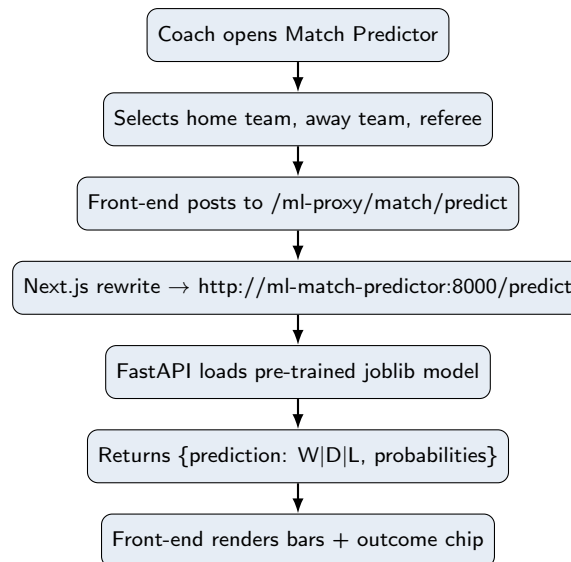


Figure 3.4: AI match prediction workflow. The proxy hop is what makes the browser see the call as same-origin.

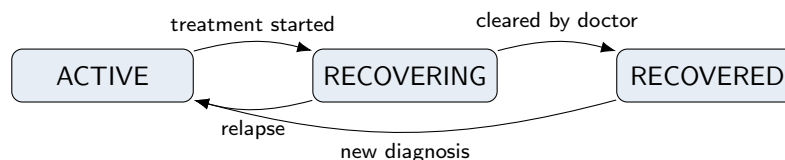


Figure 3.5: Injury status state machine (`InjuryStatus` enum in `medical-fitness`).

3.5.3 Refresh-token flow

Keycloak issues both an `access_token` (short-lived, default 7 days in `realm.json`) and a `refresh_token` (long-lived). When the access token expires, the front-end transparently calls Keycloak's `token` endpoint again with `grant_type=refresh_token` to obtain a fresh access token without prompting the user. The refresh token itself is rotated on each call to mitigate replay attacks.

3.5.4 Role-based access control

RBAC is enforced at two layers:

1. **Edge enforcement** in Next.js middleware (`src/lib/permissions.js`): every dashboard route is mapped to an array of allowed roles; the request is short-circuited with a redirect when the role is missing.
2. **Server-side enforcement** in Spring controllers: each endpoint is annotated with `@PreAuthorize` expressions that read the JWT's realm-role claim.

The two layers use *the same role list*, fed by a single source of truth (the realm JSON for Keycloak, the permissions map for the front-end), which makes role drift impossible by construction.

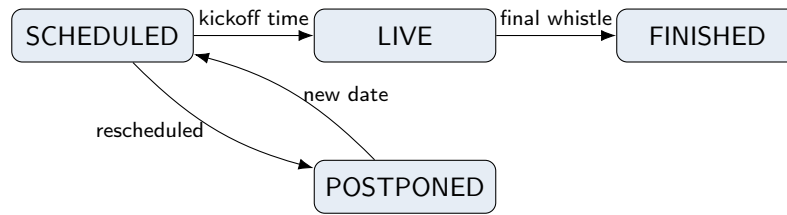
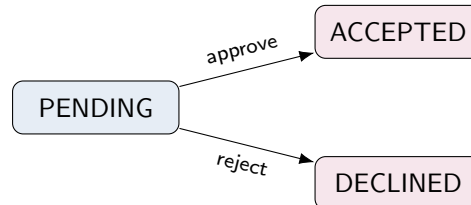
Figure 3.6: Match status state machine (`MatchStatus` enum).

Figure 3.7: Transfer request state machine. Once approved or declined, the row is immutable (a new transfer would be a new row).

3.5.5 Read-only mode for unprivileged roles

The `fan` role is a special "consumer" role: it can view Players, Teams, Matches, Training, Reports and the Dashboard, but every form's *Add*, *Edit* and *Delete* action is suppressed in the UI by the `useRole()` hook, which returns `readOnly: true` for roles listed in `READ_ONLY_ROLES`. This is purely a UX hint — the gateway still rejects any write attempt at the API level.

A Mermaid version of this sequence diagram, together with the refresh-token flow, is provided in `AUTH_SEQUENCE.md` for re-rendering.

3.6 Deployment View

This section documents how the logical microservices are mapped to physical Docker containers, ports and networks. The whole stack is launched by one command on a developer laptop or production host:

```
docker compose -f docker-compose.full.yml up -d
```

3.6.1 Container Topology

Figure 3.10 maps every service in the Compose file to its container, its image source, and its exposed host port (where applicable). All containers join a single bridge network named `mscms-network`, which means service discovery via container name works out of the box.

3.6.2 Build vs. Pull Strategy

The Compose file specifies both an `image:` and a `build:` directive for every business microservice. The dual directive lets contributors choose:

- `docker compose up` (no build) – pulls the pre-built image from GHCR (`ghcr.io/tefaa1/mscms/*`). Fast for first-time evaluators.

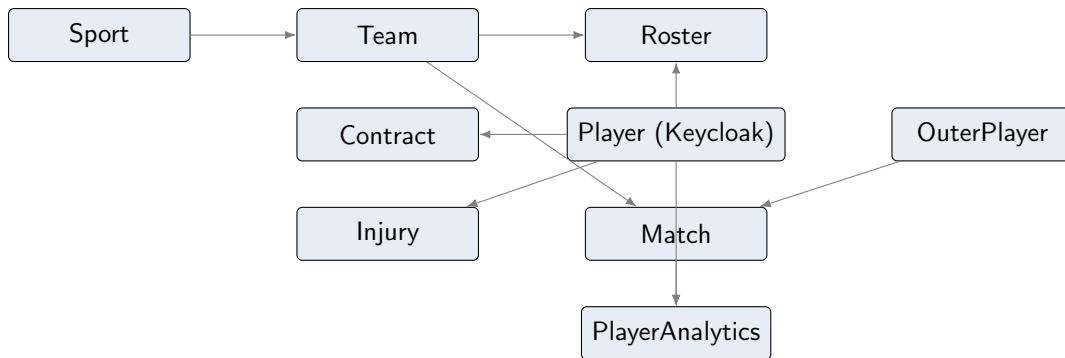


Figure 3.8: Principal entity relationships across the player, training-match, medical and reports databases.

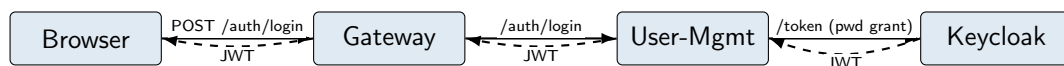


Figure 3.9: Login sequence. Solid arrows are requests; dashed arrows are responses carrying the JWT.

- `docker compose build` – builds from the local `Dockerfile` per service. Required after editing Java source. Each service’s `Dockerfile` is a multi-stage build that produces a slim JRE image.

3.6.3 Volumes and Persistence

Three named volumes survive container restarts:

- `postgres_mscms` – holds all six MSCMS databases. Wiping it (`down -v`) is the only safe way to reset seed data.
- `keycloak_data_mscms` – holds the imported realm, Keycloak’s internal H2 (or external) DB, and any users created through the admin console.
- `redis_data_mscms`, `zookeeper_data_mscms` and `zookeeper_log_mscms` – supporting infrastructure.

3.7 System Flowcharts

The flowcharts in this section describe the control flow of the most common multi-step operations across roles. They are deliberately simpler than full activity diagrams — they trace the happy path and the most important branch, so the reader can grasp the shape of each workflow before drilling into the per-role sequence diagrams in Chapter 4.

3.7.1 Admin Flow: User Provisioning

Figure 3.11 captures the admin workflow for provisioning any of the nineteen roles.

3.7.2 Head Coach Flow: Plan a Training Session

Figure 3.12 sketches the coach’s training authoring loop.

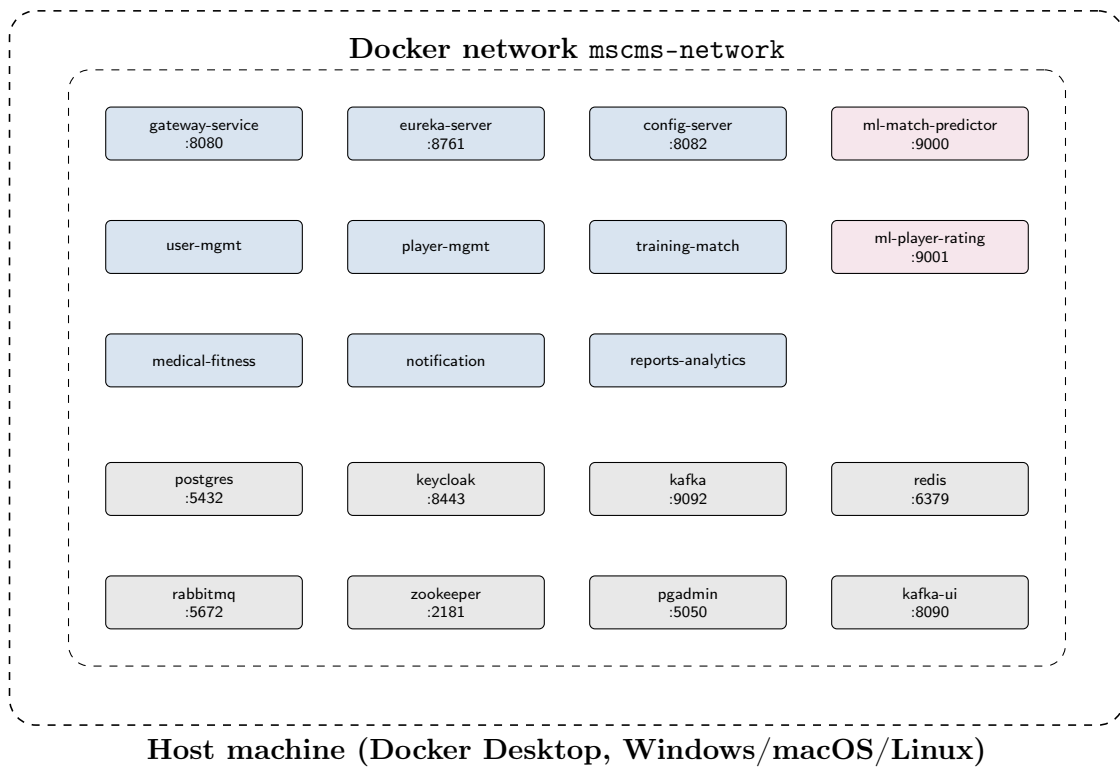


Figure 3.10: Deployment topology. Boxes in blue are Java/Spring Boot services, red are Python/FastAPI ML services, grey are infrastructure containers. All boxes share one Docker bridge network.

3.7.3 Sponsor Flow: Submit & Track an Offer

Figure 3.13 traces the sponsor side of the offer-approval workflow.

3.8 Class Diagrams

This section presents the core domain classes of MSCMS as UML-style class diagrams. Each box lists the class name (top), its key attributes (middle) and the most important methods or relationships (bottom). Bidirectional arrows mark JPA associations; the arrow direction reflects ownership.

3.8.1 User & Identity

Figure 3.14 shows how a single Keycloak identity becomes the anchor for one or more profile rows across the system.

3.8.2 Squad & Match Operations

Figure 3.15 models the squad / fixture domain served by `player-management-service` and `training-match-service`.

3.8.3 Medical Domain

Figure 3.16 shows the medical entity graph from `medical-fitness-service`.

3.9 Detailed Entity-Relationship Diagram

The high-level entity sketch in Section 3.4 shows the principal relationships across services. Figure 3.17 expands that with cardinalities and the cross-service `keycloakId` bridges that thread through every microservice’s local schema.

The cardinalities tell the story of three design decisions:

- **Keycloak as the identity bridge.** Every cross-service reference to a person uses the `keycloakId` string rather than a numeric FK. This means the `player_profile` row in `mscms_user` and the `injuries` row in `mscms_medical` can be joined logically without ever needing a cross-database constraint.
- **Roster as the season anchor.** A player can be on multiple rosters across seasons (`:*` through the `rosters` junction), but only one contract per roster (`1:1`) — which keeps the legal contract document tied to a specific season and team.
- **Outer entities mirror our own.** The `outer_teams` / `outer_players` tables (not shown above to keep the figure readable) replicate the shape of `teams` / `player_profile` so that transfers can move data symmetrically between the two sides.

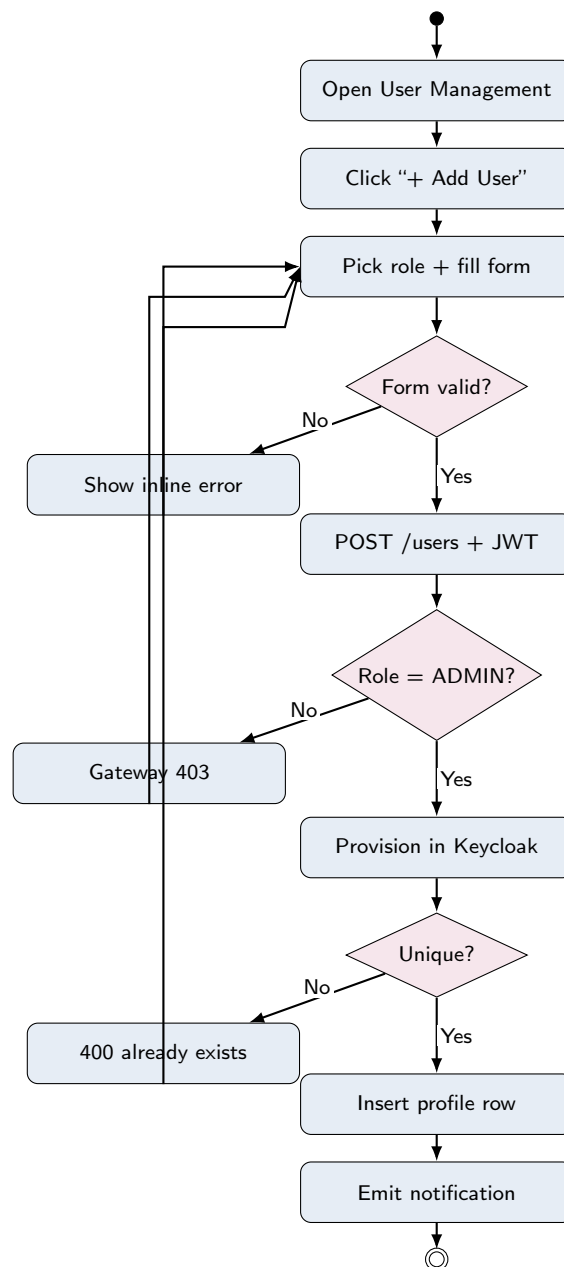


Figure 3.11: Admin flowchart for user provisioning. Three failure modes (validation, authorisation, duplicate) all loop back to the form so the admin can correct the input.

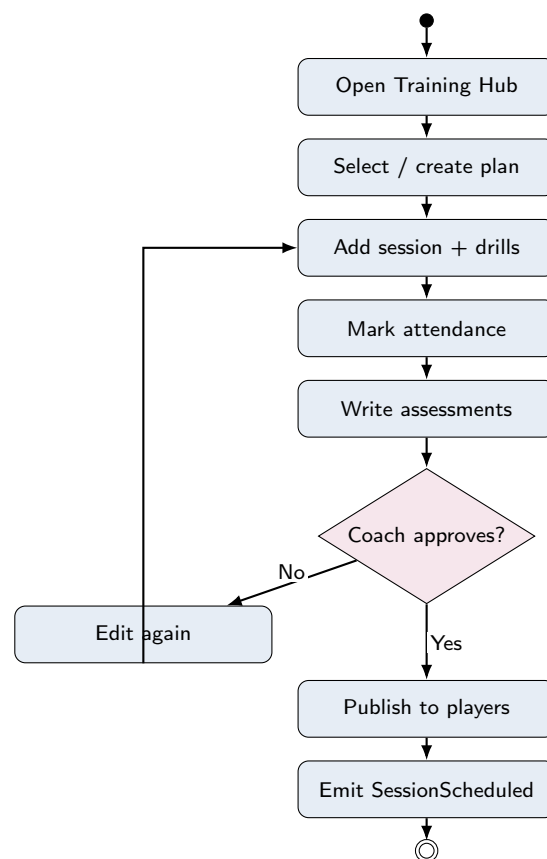


Figure 3.12: Head Coach flowchart. The coach drafts session content, marks attendance, and either approves (publishing the session and triggering a notification) or loops back to edit.

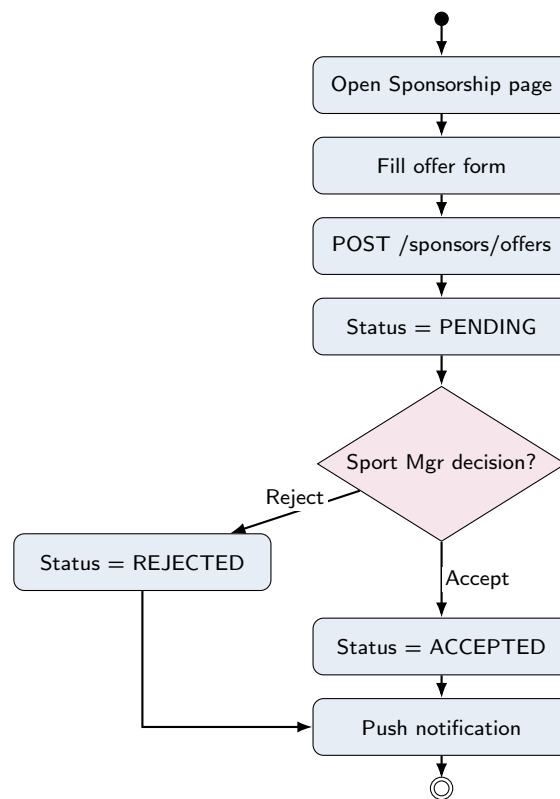


Figure 3.13: Sponsor flowchart. The sponsor submits an offer which sits in **PENDING** until the sport manager accepts or rejects it; either decision triggers a notification.

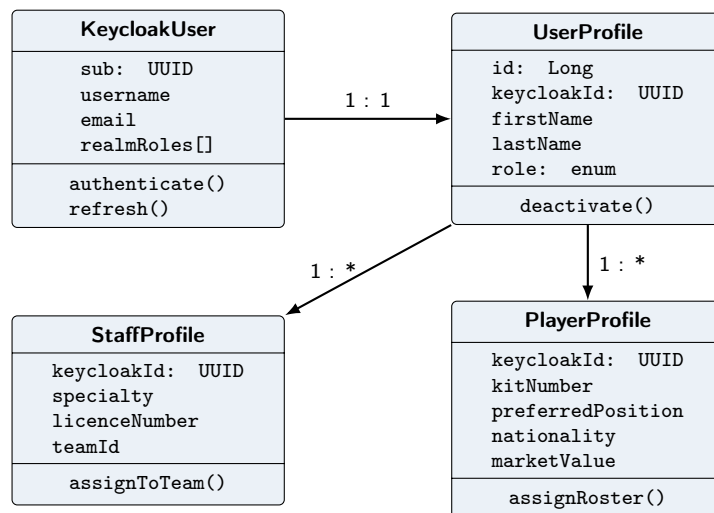


Figure 3.14: User identity class diagram. One Keycloak identity maps to one **UserProfile**, which may have either a player or staff profile attached depending on role.

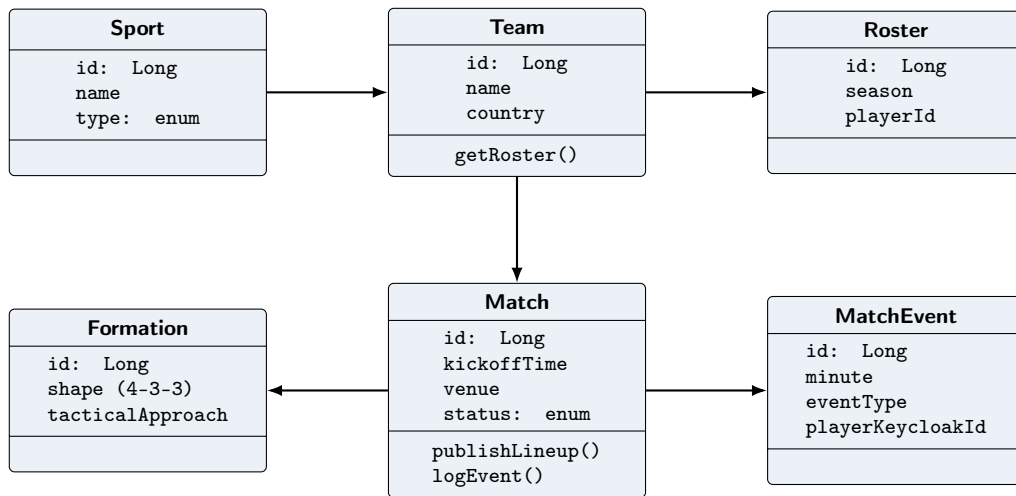


Figure 3.15: Squad and match-operations class diagram. A **Sport** contains **Teams**; each team has a **Roster** per season and plays **Matches** that produce **MatchEvents** under a chosen **Formation**.

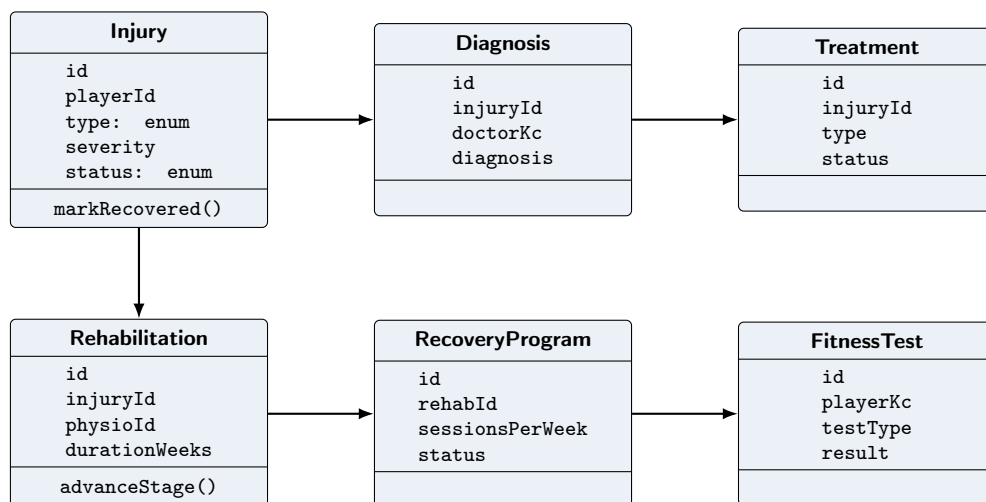


Figure 3.16: Medical class diagram. An **Injury** owns one or more **Diagnosis** and **Treatment** entries; a **Rehabilitation** can in turn drive a structured **RecoveryProgram** alongside ad-hoc **FitnessTest** measurements.

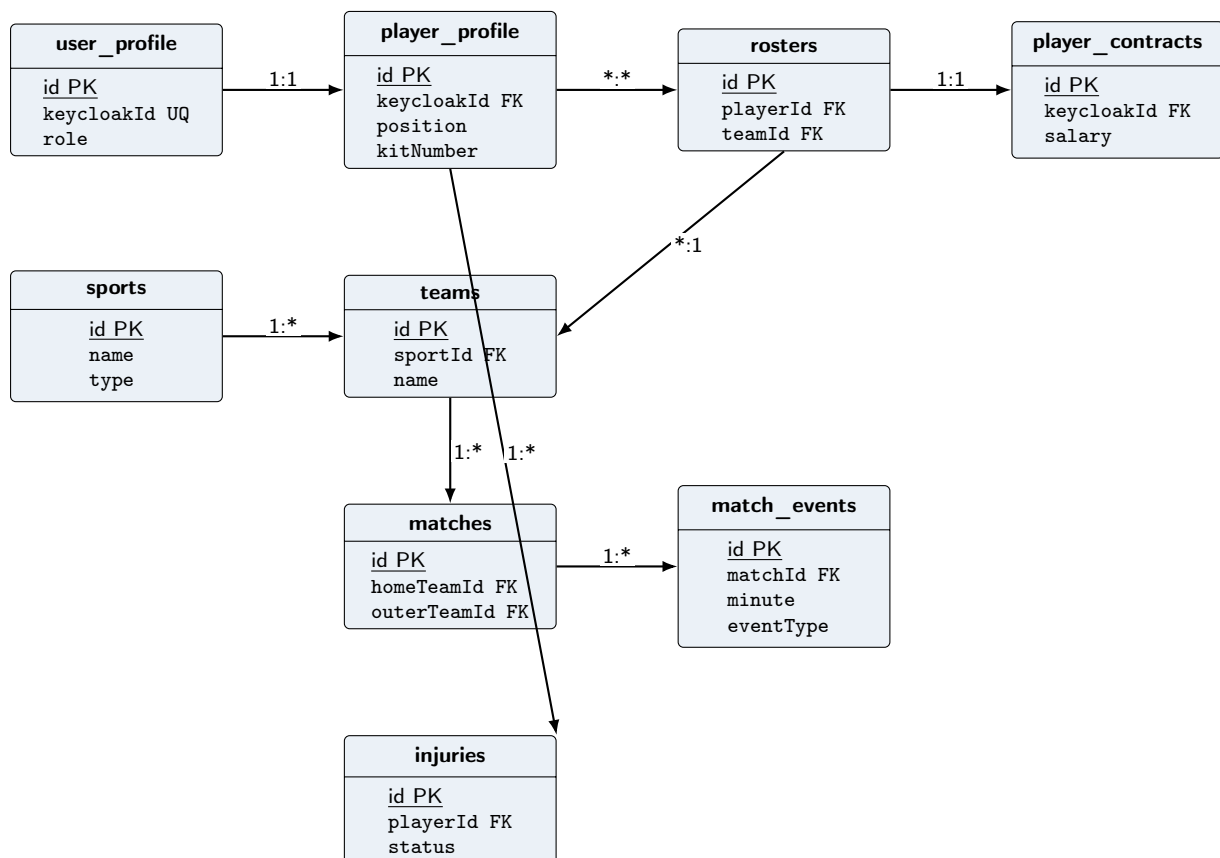


Figure 3.17: Detailed ERD across the six databases. `keycloakId` is the cross-service bridge — it lives unmodified in every schema that needs to reference a person. Cardinalities use crow-foot conventions (1:1, 1:*, *:*)).

Chapter 4

Sequence Diagrams

This chapter collects every *behavioural* model of MSCMS in one place: per-role operation sequences (Admin, Sport Manager, Team Manager, Head Coach, Doctor, Scout, Sponsor, Performance Analyst, Player, Fan), authentication and refresh-token flows, machine-learning prediction call paths, and the cross-cutting notification fan-out. Static structural views live in the previous chapter (Chapter 3). The diagrams use TikZ lifelines to render the end-to-end flow; each diagram lists the participating actor on the left and the participating services as vertical lifelines across the page.

4.1 Diagram Notation

Every sequence diagram in this chapter follows the same visual conventions:

- **Lifelines.** The boxes along the top edge are the participants — actor on the left, then the services involved from left to right in roughly the order they are contacted. The dotted vertical lines underneath each box are the lifelines that anchor every message in time.
- **Solid arrows** represent *requests* — synchronous method calls or HTTP requests where the sender expects a response.
- **Dashed arrows** represent *responses* — the return value of a request, propagated back along the call stack.
- **Vertical position** is time: messages higher up happen before messages lower down. There is no implied unit of duration, only relative ordering.
- **Labels** on each arrow give the HTTP verb plus path (POST /injuries, GET /players/{id}, etc.) or the method name for in-process calls.
- **Asynchronous fan-out** (Kafka events, notification broadcasts) is shown as a separate arrow originating after the synchronous request has already completed, so the reader can see that the caller did not block on the fan-out.
- **Auth bookkeeping** (gateway JWT verification, role propagation as headers) is shown as a short label on the forwarding arrow rather than a separate trip; this keeps each diagram readable while still making the security boundary visible.

Where a particular call path is complex enough to obscure the higher level intent, a short “How it works” paragraph follows the figure walking through the steps in plain prose.

4.2 Per-Role Sequence Diagrams

Admin: Create a Staff User

Figure 4.1 shows the path of a *Create staff user* request from the admin browser to Keycloak and back into the application’s profile tables.

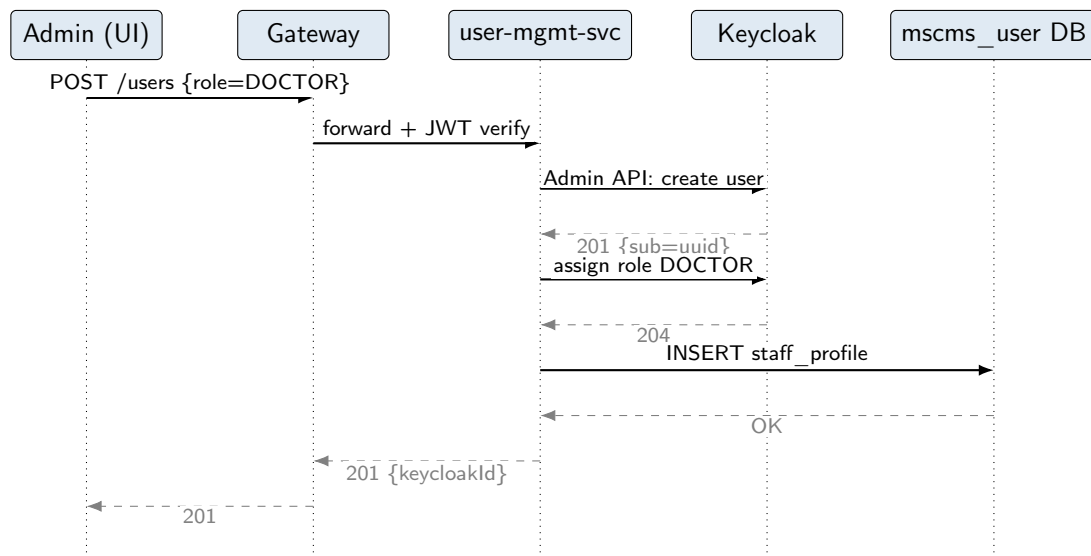


Figure 4.1: Admin creates a staff user. Keycloak is the source of truth for identity; the profile row in `mscms_user` is written only after Keycloak commits.

How it works. The admin fills the *New User* modal and submits. The browser sends a `POST /users` carrying the JSON body and an `Authorization: Bearer` header. The gateway verifies the signature against Keycloak’s JWKS, checks that the caller’s realm role is `ADMIN`, and forwards the request to `user-management-service`. The service calls Keycloak’s Admin API to provision the account (which generates the `sub` UUID) and then assigns the requested role (`DOCTOR` in this example). Only after Keycloak returns the final 204 does the service open a transaction on `mscms_user` and insert the corresponding `staff_profile` row keyed by that UUID. If any of the three calls fail, nothing is persisted on either side and the front-end shows a toast with the precise error code returned by the gateway.

Admin: Broadcast a Notification

How it works. The broadcast modal collects the message body, the priority level (low / normal / high / critical) and the target audience (one role group, one user, or system-wide). The front-end posts to `POST /notifications/broadcast`. The gateway validates the JWT and forwards. The notification service does *not* fan out the notification itself on the request thread — instead it publishes a single `NotificationBroadcast` event onto Kafka and immediately returns 202 Accepted. A Kafka consumer running inside the

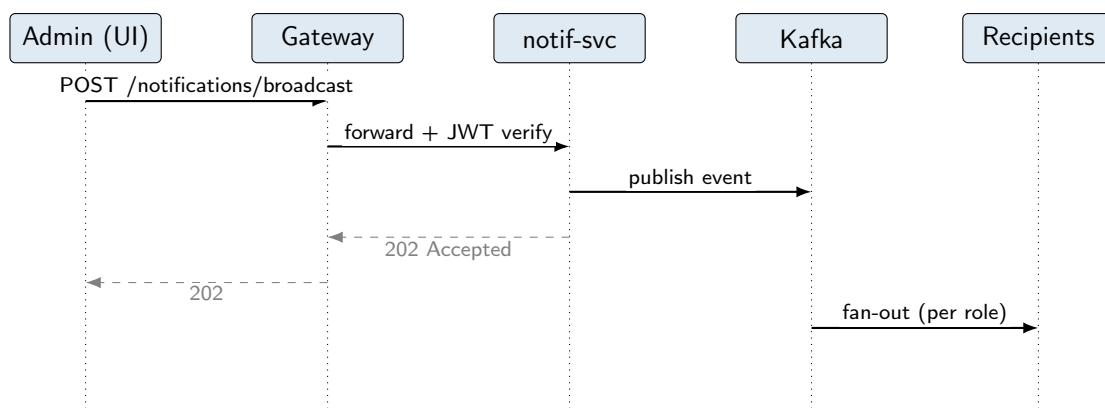


Figure 4.2: Admin broadcasts a notification. The HTTP response returns immediately; Kafka takes care of asynchronous fan-out.

same service then resolves the target role list against `mscms_user`, writes one row into `notifications` per recipient, and (for priorities `HIGH` or `CRITICAL`) issues SMTP sends through `notification-mail-service`. Because the synchronous response returns before fan-out completes, the admin UI feels instant even when broadcasting to hundreds of users.

Sport Manager: Approve a Sponsor Offer

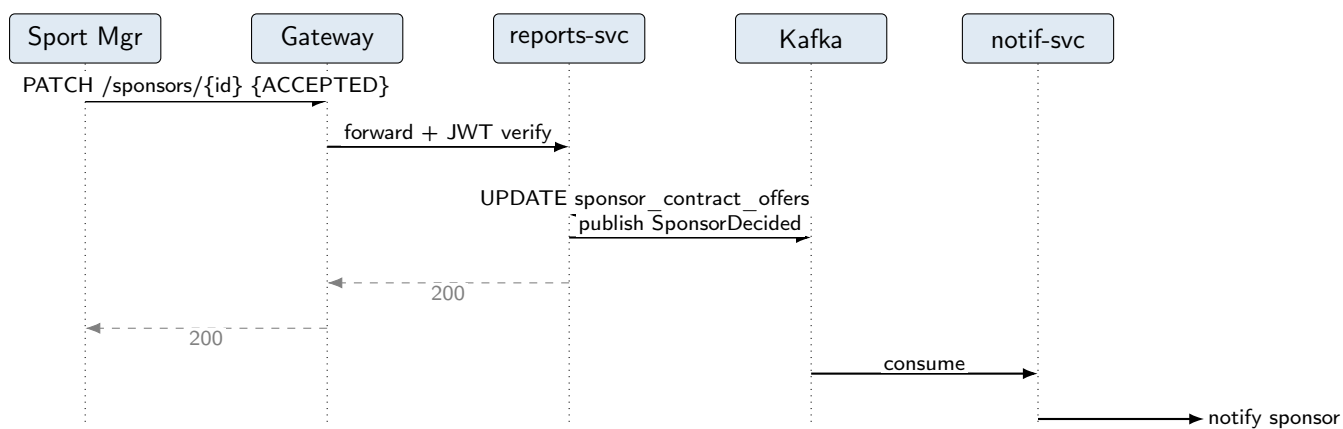


Figure 4.3: Sport manager accepts a sponsor offer. The decision both updates the database row and emits a Kafka event consumed by the notification service.

How it works. The sport manager opens the Sponsorship page, sees the pending offer in the `PENDING` column, and clicks *Accept*. The front-end sends a partial update via `PATCH /sponsors/{id}` with body `{status: "ACCEPTED"}`. The gateway propagates the JWT and forwards to `reports-analytics-service`, which updates the `sponsor_contract_offers` row inside a transaction and publishes a `SponsorDecided` event on Kafka. The synchronous response returns immediately to the sport manager so the UI flips the chip from `PENDING` to `ACCEPTED` without waiting on the notification fan-out. Downstream, the notification service consumes the event and pushes an in-app notification (and an optional email) to the originating sponsor account so they know their offer was approved.

Team Manager: Create a Match

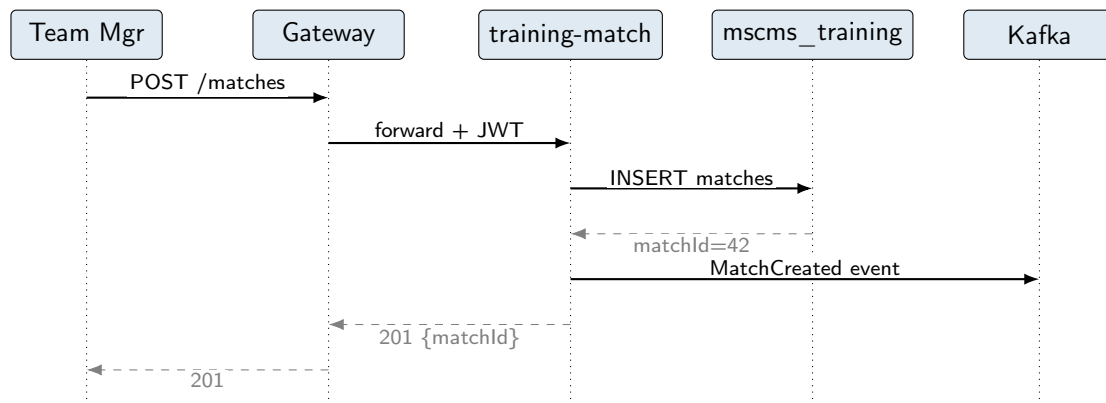


Figure 4.4: Team manager creates a match. A `MatchCreated` event on Kafka triggers downstream notifications and analytics pre-aggregation.

How it works. The team manager opens *New Match* and fills opponent, venue, kick-off time and competition. The form posts to `POST /matches`. The gateway forwards to `training-match-service`; the service starts a transaction, inserts the row into the `matches` table, and emits the generated `matchId` back through the chain. Before the transaction commits, the service publishes a `MatchCreated` event on Kafka carrying the new `matchId` and the originating team. Two downstream consumers pick this event up: the notification service announces the fixture to coaches and players, and the reports-analytics service pre-creates a placeholder `team_analytics` row scoped to the new match so post-game data writes are $O(1)$ inserts rather than upsert lookups.

Head Coach: Create a Training Session

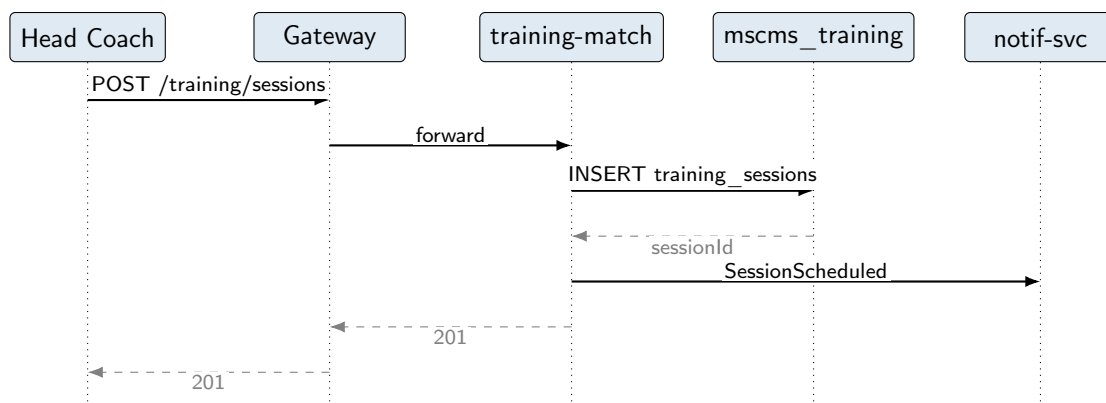


Figure 4.5: Head coach schedules a training session. Players receive a `SessionScheduled` notification through the same fabric used for match line-ups.

How it works. The head coach picks a date and time, attaches one or more drills from the drill library, and clicks *Schedule*. The request lands at the gateway, which propagates the JWT to `training-match-service`. The service performs three writes inside a single transaction: the new row in `training_sessions`, the join rows linking that session to the chosen drills (`training_drills`), and a placeholder attendance row per rostered player.

As the transaction commits, the service publishes a `SessionScheduled` event picked up by the notification service, which delivers in-app notifications to every player on the roster.

Doctor: Report an Injury

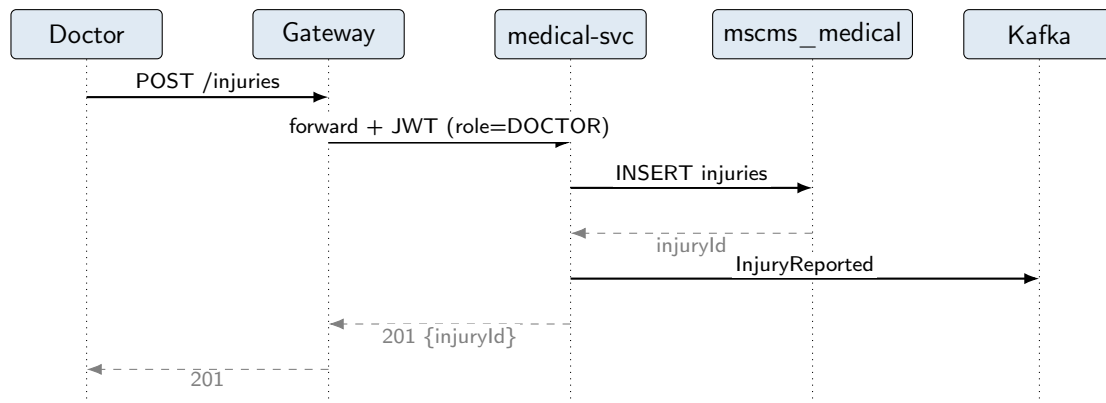


Figure 4.6: Doctor reports an injury. The `InjuryReported` Kafka event is consumed by the notification service (which alerts the head coach and team manager) and by the reports service (which recomputes player availability).

How it works. The doctor opens the Injuries page, selects the affected player, fills in the body part, severity and a free-text description, and clicks *Save*. The browser sends `POST /injuries`. The gateway verifies that the caller's JWT carries the `DOCTOR` role and forwards to `medical-fitness-service`, which writes a row into `injuries` with status `ACTIVE`. Before the transaction commits, the service publishes an `InjuryReported` event on Kafka. Two consumers downstream: `notification-mail-service` fans out an alert to the head coach, team manager and player; and `reports-analytics-service` flips the player's availability flag so any subsequent line-up tool will exclude them by default. The synchronous `201` returns to the doctor immediately so the UI doesn't block on the event-driven side effects.

Scout: Submit a Scout Report

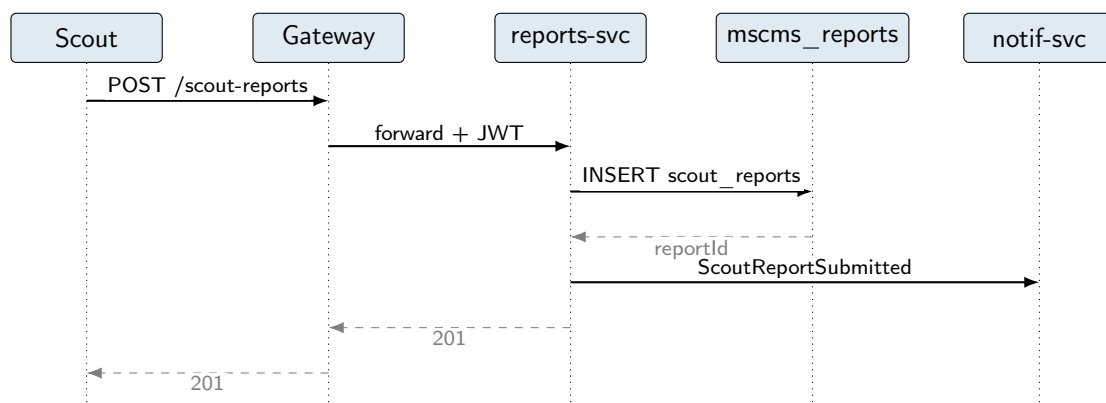


Figure 4.7: Scout submits a structured report. The notification service delivers the report to the sport manager's transfers inbox.

How it works. The scout opens an outer player profile and clicks *New Report*. They fill in the five-axis rating (technical, physical, tactical, mental, potential), the strengths/weaknesses text, and a recommendation chip (SIGN, MONITOR, REJECT). On submit, the front-end sends a POST `/scout-reports` to the gateway. After JWT verification, the request reaches `reports-analytics-service`, which writes the row into `scout_reports` and publishes a `ScoutReportSubmitted` event. The notification service delivers the report into the sport manager’s transfers inbox so it can be cross-referenced with any active transfer negotiation for the same player.

Sponsor: Submit an Offer

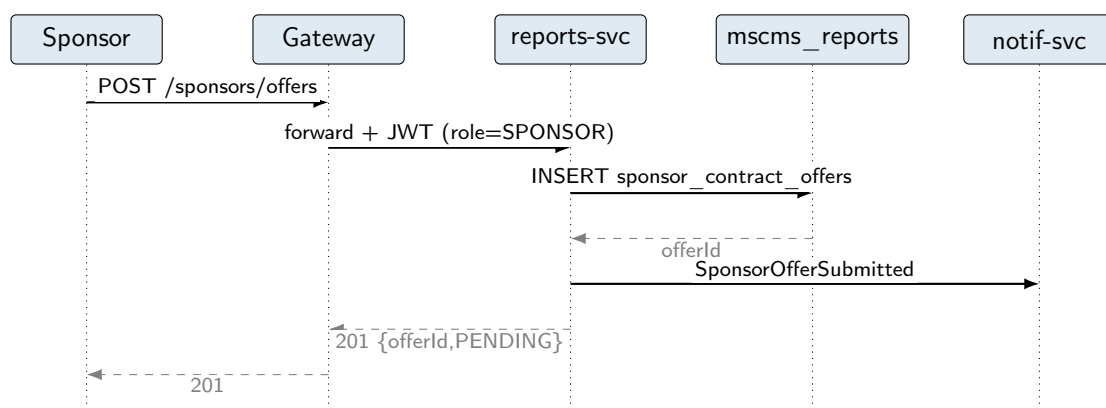


Figure 4.8: Sponsor submits an offer that enters the PENDING state. The notification service alerts the sport manager for a decision.

How it works. The sponsor enters the offer value, duration in months, and optional sport-targeting (whole club vs. football vs. basketball, etc.) and submits. The gateway accepts the request only if the caller’s JWT carries the SPONSOR role. The reports-analytics-service writes the row into `sponsor_contract_offers` with status PENDING and emits `SponsorOfferSubmitted`. The notification service delivers an in-app alert to the sport manager, and the front-end flashes the pending count on the sport manager’s dashboard the next time it loads. This is the mirror image of the approval flow shown in Figure 4.3: the sponsor pushes the offer into the pending state, the sport manager later pulls it out into ACCEPTED or REJECTED.

Performance Analyst: AI Player Rating

How it works. The analyst types a player name in the autocomplete (which is bounded to twenty known players from the training dataset, so the input cannot produce a *not found* error). On submit, the front-end calls GET `/ml-proxy/player/predict/{name}`. The Next.js dev server (or the production reverse proxy in deployment) rewrites this path to `http://ml-player-rating:9001/predict/{name}`, making the call same-origin from the browser’s perspective and avoiding a CORS pre-flight. The FastAPI service loads the player’s feature row from `Football_Players_Data.csv`, feeds it through the joblib regression model, and returns both the predicted rating and the actual rating present in the dataset (so the analyst can sanity-check the prediction’s accuracy on known data).

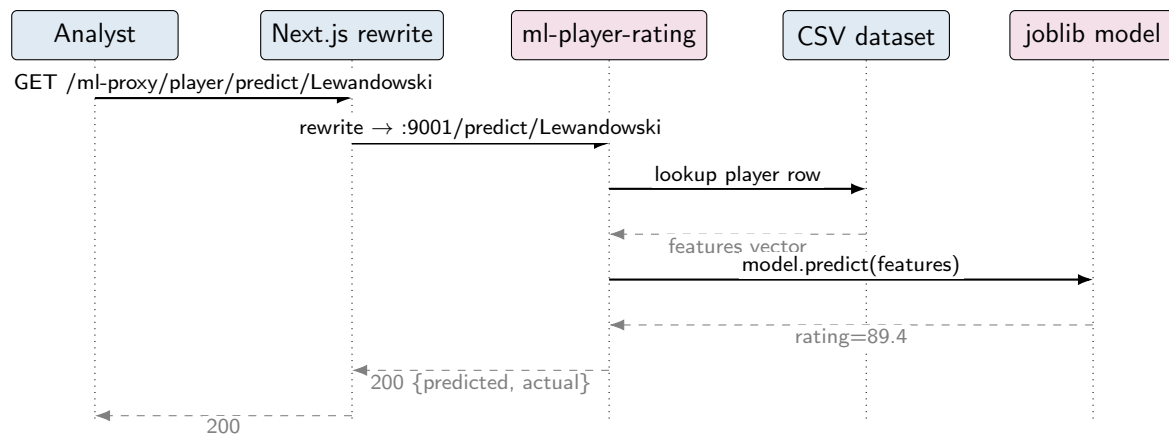


Figure 4.9: AI player rating request. The Next.js rewrite proxy makes the call same-origin from the browser’s perspective; the FastAPI service performs the lookup and prediction.

Player: View Own Performance

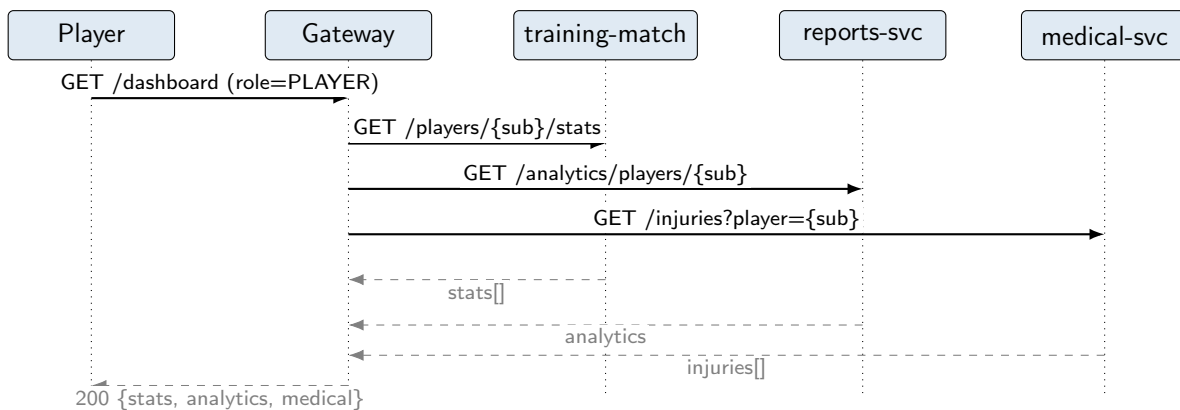


Figure 4.10: Player dashboard fan-out. Three downstream services are queried in parallel; the gateway composes the response so the browser sees a single request.

How it works. When a player loads `/dashboard`, the front-end issues a single bundled request to the gateway. The gateway recognises the dashboard route, decodes `sub` from the JWT, and dispatches three parallel requests internally: per-match stats from `training-match-service`, analytics roll-up from `reports-analytics-service`, and current injury status from `medical-fitness-service`. As each downstream response arrives, the gateway accumulates it into a single composite payload and returns a single 200 to the browser. The player never sees how many services were involved — they see one network request and one render pass.

Fan: Browse Public Match Calendar

How it works. A fan navigates to `/dashboard/matches`. Before the request leaves the browser, the Next.js middleware calls `canAccess(role=“fan”, path)` from `permissions.js` — the fan is in the allowed-roles list for this route, so the request proceeds. The gateway accepts the JWT and forwards a read-only `GET /matches` to `training-match-service`. The same React component used by team managers renders the response, but the `canAccess`

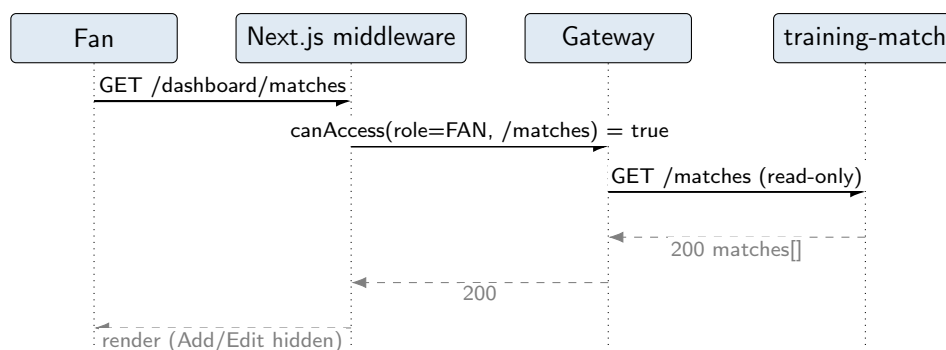


Figure 4.11: Fan browses the public match calendar. The middleware grants the route and the page is rendered with mutation controls hidden by `canAccess`.

helper returns `false` for any mutation route, so the *Add*, *Edit*, and *Delete* buttons are not rendered. The fan sees a clean read-only fixture list without ever knowing that the same React component handles the staff view too.

Cross-cutting: Notification Fan-out on Domain Event

Figure 4.12 shows the most distinctive cross-cutting pattern in MSCMS: any domain event (injury, sponsor decision, transfer, match created) is dispatched by Kafka and delivered both in-app and (optionally) by email.

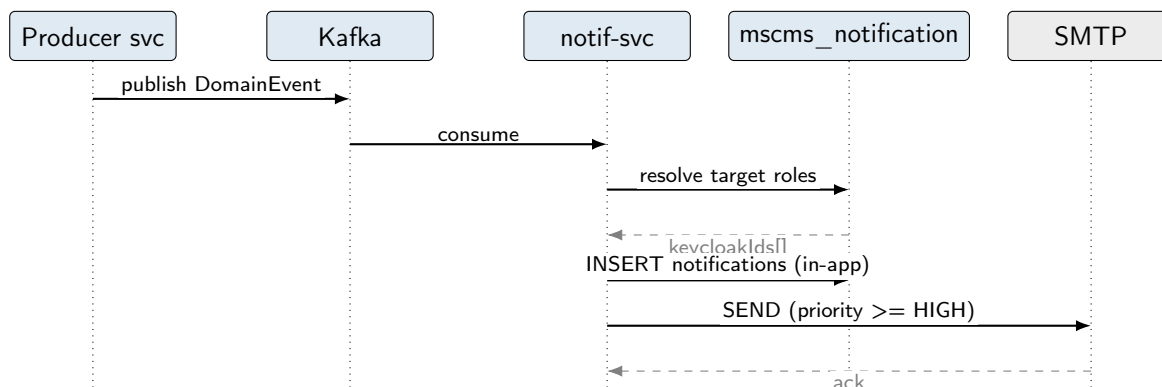


Figure 4.12: Notification fan-out. A single Kafka event becomes both an in-app notification row and an outbound email, with email restricted to priority `HIGH` or `CRITICAL`.

How it works. Any domain service (medical, training, reports, player-management, etc.) can publish a structured `DomainEvent` on Kafka without owning a recipient list. The notification service is the only consumer of that topic. On receipt, it resolves the event’s *target role* (e.g. “head coach of team 1” or “all sponsors”) against `mscms_user` to produce the concrete `keycloakId` list. For every recipient, it inserts one row into the `notifications` table with a short `title`, a `body`, the originating `eventType` and the configured `priority`. If `priority ≥ HIGH`, an SMTP send is issued through `notification-mail-service`. The in-app channel and the email channel run independently — if SMTP fails, the in-app notification still lands; if the database transaction fails, no email is sent. This separation is why the notification fabric can absorb 100x the traffic of any single domain service: producers fire and forget, the consumer handles fan-out at its own pace.

4.3 Authentication & Session Sequences

The authentication-related sequences span the gateway, the `user-management-service` and Keycloak. They are kept in their own section because they apply to every other operation in the system — every authenticated request implicitly depends on the login and refresh flows shown here.

Login Flow with JWT Issuance

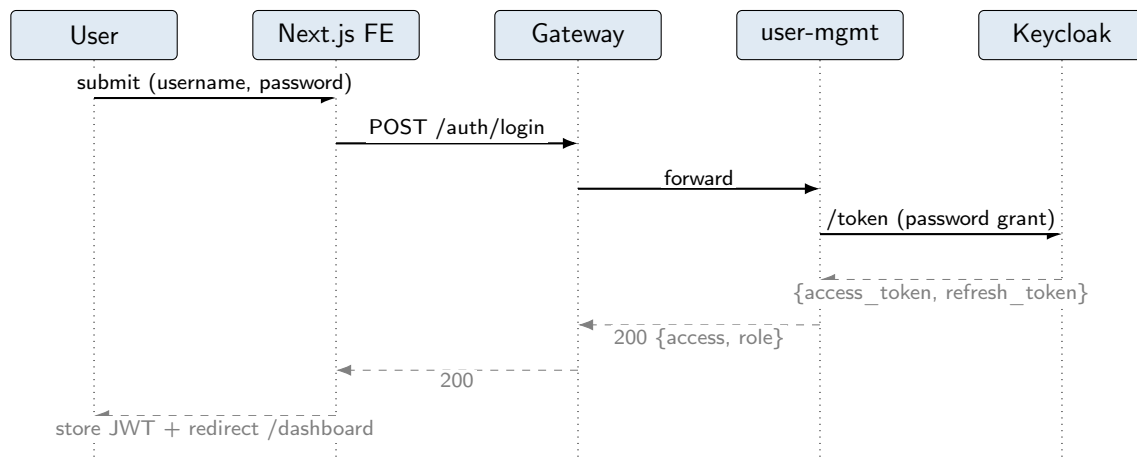


Figure 4.13: Login flow with JWT issuance. Keycloak owns the credentials; the front-end never sees the password after it leaves the submit handler.

How it works. The user submits credentials. The Next.js front-end posts to `/auth/login` on the gateway, which forwards to `user-management-service`. That service translates the request to Keycloak’s OAuth 2.0 password grant. Keycloak verifies the credentials, issues an `access_token` (short-lived) and a `refresh_token` (long-lived), and returns them. The `user-management-service` extracts the user’s primary realm role from the JWT claims, attaches it as a flat `role` field in the response (so the front-end doesn’t need to decode the JWT just to know the role), and returns. The front-end stores the access token in `localStorage`, the role both in `localStorage` and a same-name cookie consumed by the Next.js middleware, and redirects to `/dashboard`.

Refresh-Token Flow

How it works. The access token has a short TTL (one hour by default in the seeded realm). When it expires, the next API call returns 401. The front-end’s API interceptor catches this, calls `POST /auth/refresh` with the `refresh_token` from `localStorage`, and waits for the new token pair. Keycloak issues a new access token and rotates the refresh token (so a stolen refresh token cannot be used twice). The interceptor stores both, then transparently retries the original failed request. From the user’s perspective, the only visible artefact of the entire round trip is a small loading spinner. If the refresh token has been revoked — for example after a sign-out from another device — the refresh call returns 400 `invalid_grant`; the interceptor then clears storage and redirects to `/login`.

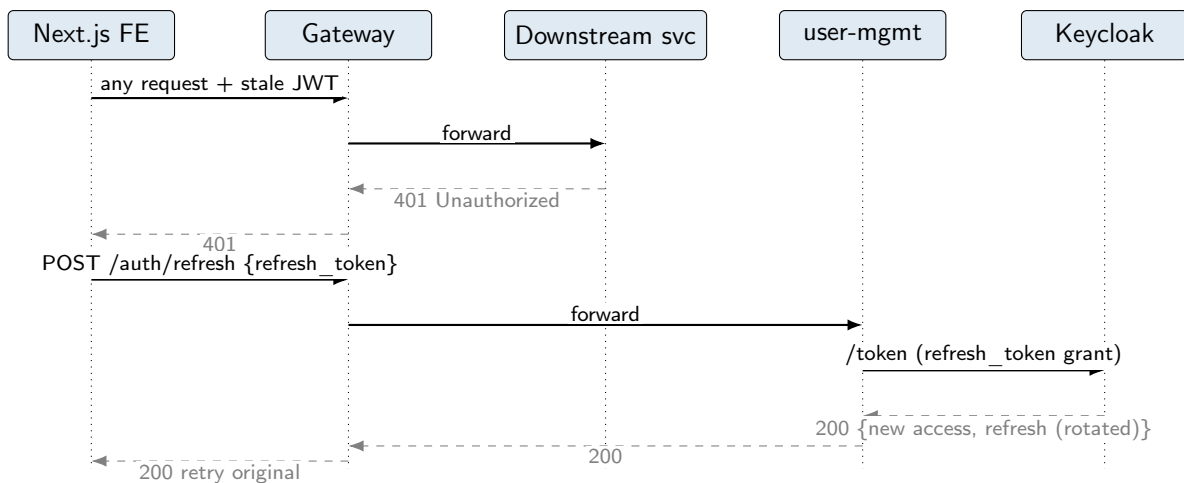


Figure 4.14: Refresh-token flow. A 401 triggers a transparent token refresh and the original request is retried; the user is never prompted to log in again unless the refresh token itself is revoked.

4.4 Operational Flow Extensions

The remaining diagrams cover operations that span more than one service and are not captured by the per-role sequences above. They illustrate the integration points between coaches, sport managers, medical staff and the analytics layer.

AI Match Prediction

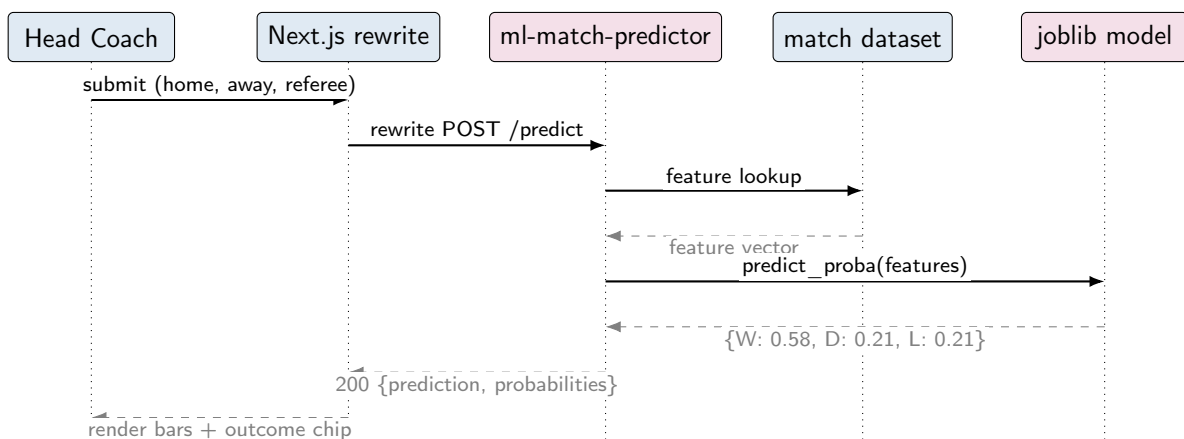


Figure 4.15: AI match prediction sequence. The same-origin Next.js rewrite keeps the browser unaware that the call leaves the front-end tier.

How it works. The coach picks home team, away team and a referee (with an *Other* fallback if the referee is unknown to the model). The Next.js dev server rewrites `/ml-proxy/match/predict` to `http://ml-match-predictor:9000/predict`, so the browser sees a same-origin call — no CORS pre-flight, no cookie-credentials issue. The FastAPI service looks the historical feature row up in `FCB_Football_Matches.xlsx`, runs the joblib classifier and returns both the predicted outcome (W, D, L) and the class probabilities. The

front-end renders these as proportional bars under an outcome chip, so the coach can see both the prediction and how confident the model is.

Publishing a Match Line-up

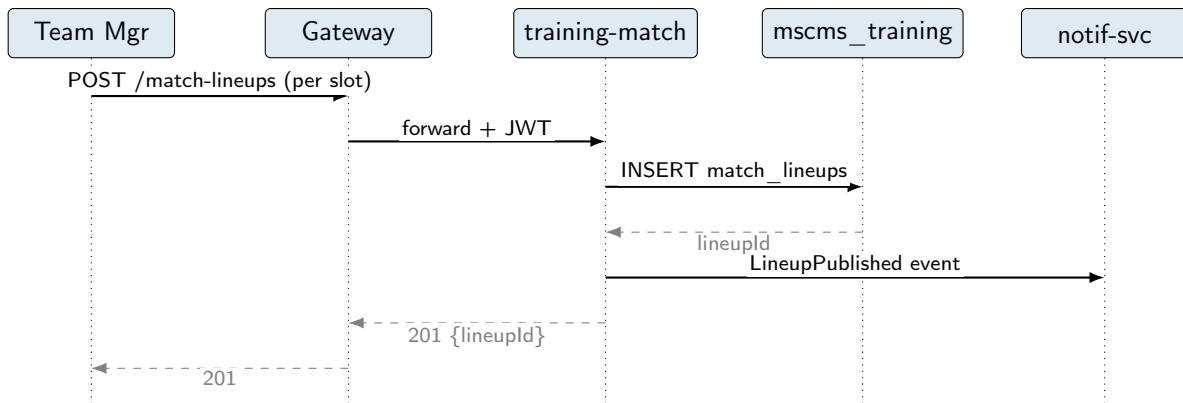


Figure 4.16: Publishing a line-up. Each slot is one insert; the final publish emits a single notification rather than one per player.

How it works. The team manager arranges players on the tactical board (drag-and-drop) under the chosen formation. Saving the draft inserts `match_lineups` rows linked to the `match_formation` record. Clicking *Publish* flips the formation's `published_at` flag and emits a single `LineupPublished` event. The notification service consumes it once and fans out a notification per affected player (rather than one notification per row inserted), keeping the alert volume sensible even for a 23-player squad. Players see a single line-up announcement in their dashboard rather than dozens of slot notifications.

Sport Manager Approves a Transfer

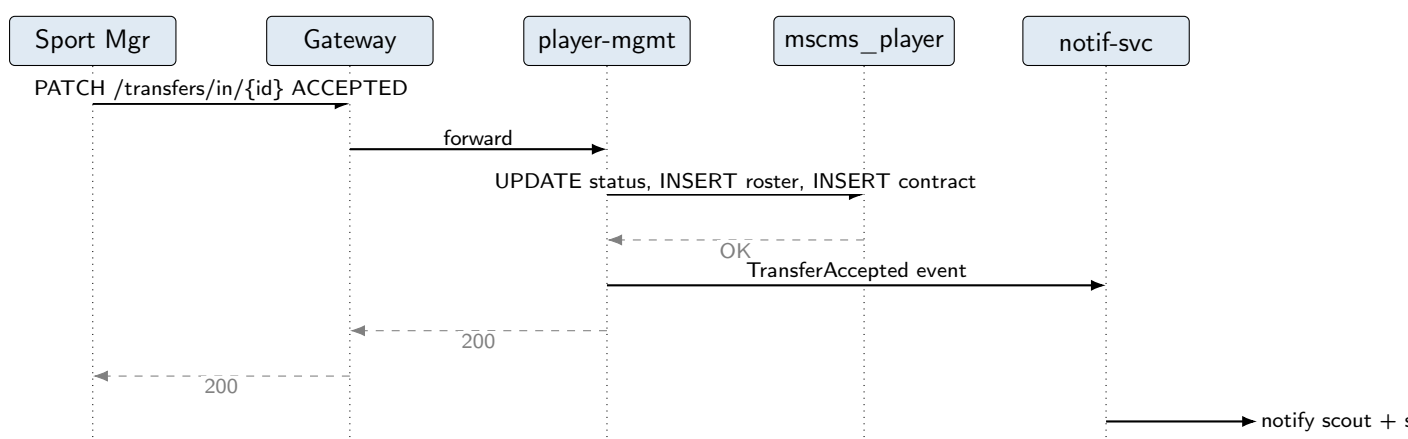


Figure 4.17: Sport manager accepts an incoming transfer. A single transaction updates the transfer row, creates the new roster entry for the incoming player and inserts their contract.

How it works. The sport manager opens the Transfers page, reviews the incoming request alongside any attached scout reports, and clicks *Accept*. The single PATCH the

front-end sends triggers three writes inside one transaction at the player-management-service: flipping the `player_transfer_incoming.status` to `ACCEPTED`, inserting a new `rosters` row that places the player on the target team for the current season, and inserting a `player_contracts` row carrying the agreed terms. If any of the three writes fails, the transaction rolls back as a unit. After commit, a `TransferAccepted` event flows to the notification service, which alerts both the originating scout (so they can close their pipeline view) and the team manager of the receiving team.

Updating a Rehabilitation Status

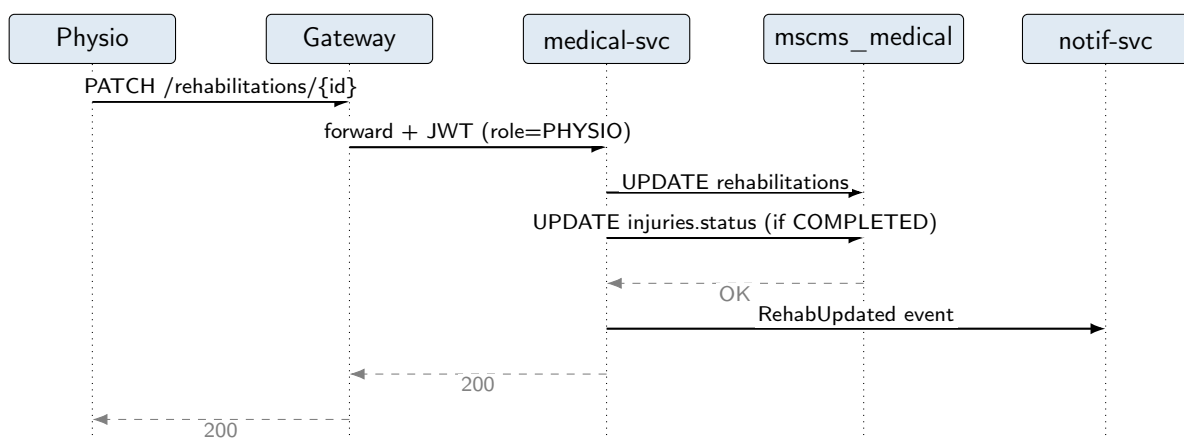


Figure 4.18: Physio updates a rehabilitation status. Marking a rehab `COMPLETED` also flips the parent injury to `RECOVERED` so the head coach’s availability view updates automatically.

How it works. The physio opens the rehab record, advances the stage (e.g. *Phase 2* → *Phase 3*), records progress notes, and saves. The medical service updates the `rehabilitations` row and — if the new status is `COMPLETED` — cascades that into the parent `injuries` row, flipping its status to `RECOVERED`. Doing both updates inside the same transaction is what guarantees the head coach’s availability view stays consistent: there is no window in which a rehab is complete but the injury is still active. A `RehabUpdated` event is emitted regardless; the notification service uses it to deliver an update to the head coach and to the player whose rehab progressed.

Chapter 5

Implementation and Results

This chapter walks through every page of the MSCMS front-end, grouped by the user role that owns it. Each subsection follows the **Description** / **Key Features** / **Result** format and ties the on-screen feature to the back-end service, database table and the Keycloak role guarding it. Screenshots are placed inline so the reader can match each feature to what the user actually sees on screen.

5.1 Onboarding Page

The onboarding surface is the public-facing landing page rendered at / when the user is not yet authenticated. It introduces MSCMS and routes the user into the login flow.

Key Features:

- Hero section themed in the modelled club’s brand colours (Barcelona blue #004D98 and red #A50044).
- Brief platform pitch summarising the eight functional domains of the system.
- Single “Login” call-to-action that links to the /login route.

Result: A first-time visitor immediately understands what MSCMS is and where to authenticate. No data is fetched on this page, keeping it fast and SEO-friendly even before login.

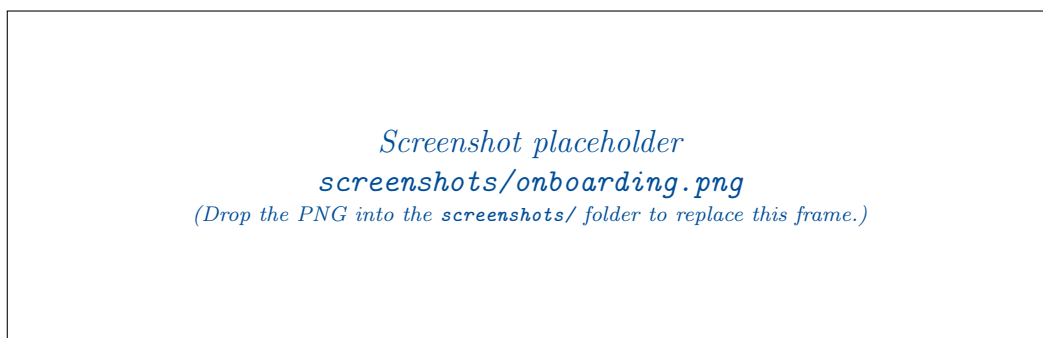


Figure 5.1: Onboarding / landing page with the Barcelona-themed hero and the *Login* call-to-action.

5.2 Login Page

Description: The login page allows any of the 19 roles to authenticate against Keycloak using the OAuth 2.0 password grant. The form validates input client-side, then posts `{username, password}` to `POST /auth/login` on the gateway, which forwards to `user-management-service` and on to Keycloak.

Key Features:

- Input validation (non-empty username, non-empty password).
- Friendly toast feedback for 401 `Unauthorized` responses.
- Decodes the returned JWT (`jwt-decode`) to store `token`, `keycloakId` (sub claim) and `user_role` (mapped from `realm_access.roles`) in `localStorage`, plus a same-name cookie for the Next.js middleware.
- Redirects to `/dashboard` on success.

Security: JWT signature is verified server-side at every subsequent request by the gateway against Keycloak’s JWKS endpoint. Username regex (`/^[a-zA-Z0-9._-]+$`) mirrors Keycloak’s own validation rules to prevent `error-username-invalid-character` responses.

Result: A reliable, role-aware single-sign-on surface that issues stateless tokens consumed identically by both the front-end middleware and the back-end gateway.

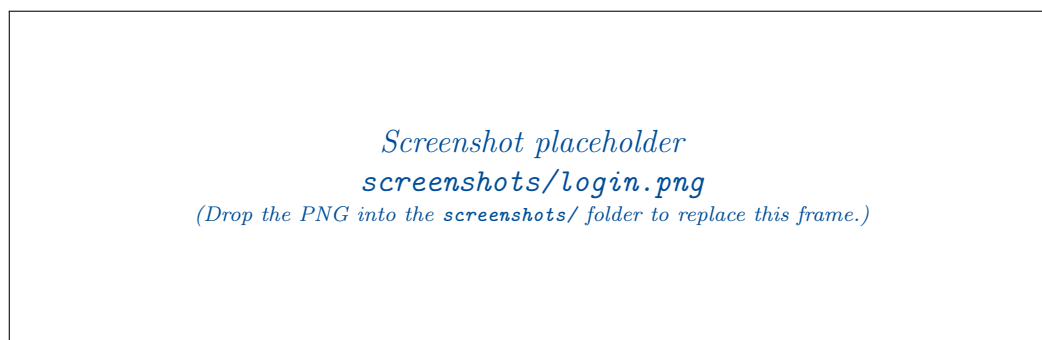


Figure 5.2: Login page. The same form is reused by every one of the 19 roles; the destination dashboard is computed from the JWT after success.

5.3 Forgot Password / Refresh-Token Flow

Description: While password reset itself is handled by Keycloak’s own account console, the front-end implements transparent session refresh: if any API call returns 401, the front-end posts `POST /auth/refresh {refresh_token}` and silently retries the original call. If the refresh token has been revoked, the user is logged out and redirected to `/login`.

Result: Long-lived sessions remain stable without forcing users to re-enter credentials every hour, while expired or revoked tokens fail safely.

5.4 Admin Role Implementation

The Admin role has the broadest reach in MSCMS. It governs system-wide user lifecycle, has read access to every operational domain and can issue broadcast notifications.

Admin Dashboard

Description: The admin landing surface aggregates counts across all six databases through parallel calls to `user-management-service`, `player-management-service`, `training-match-s`, `medical-fitness-service`, `reports-analytics-service` and `notification-mail-service`.

Key Features:

- Top-line counts: total users, total players, total teams, total matches.
- Per-discipline pie chart (football, basketball, handball, volleyball) computed from the player roster, using the `POSITION_TO_SPORT` map mirrored from the back-end `Position` enum.
- Recent activity feed sourced from the notification service.
- Quick-action chips routing to user management, staff management and notification broadcasts.

Result: The admin has a real-time pulse of the platform on a single screen, with one-click drill-down into every operational surface.

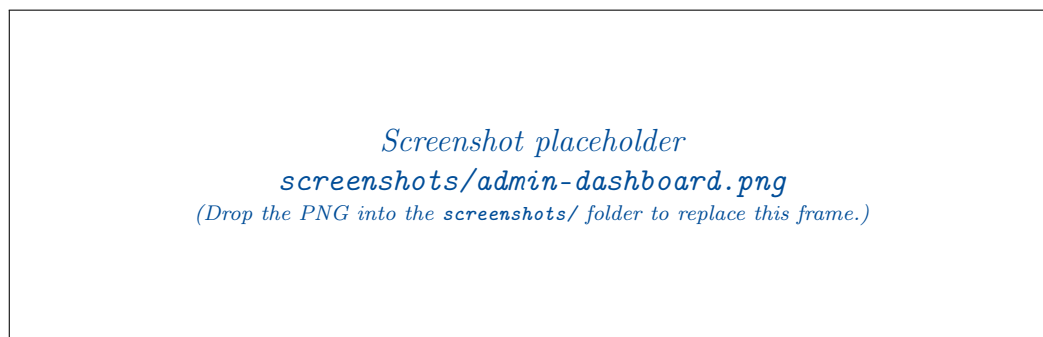


Figure 5.3: Admin dashboard aggregating counts from six microservices into a single overview.

Users Management Page

Description: CRUD over all 19 roles, surfaced through `user-management-service` which proxies the Keycloak Admin API and persists role-specific profile rows (e.g. `player_profile`, `staff_profile`) under a single `keycloakId`.

Key Features:

- Filter by role and search by username / email.
- Create user modal with role picker, username regex validation and immediate Keycloak provisioning.

- Edit profile data without losing the Keycloak identity.
- Deactivate / delete users (cascades into Keycloak realm revocation and downstream profile removal).

Result: A single source of truth for user identity that keeps Keycloak and the application databases consistent through every mutation.

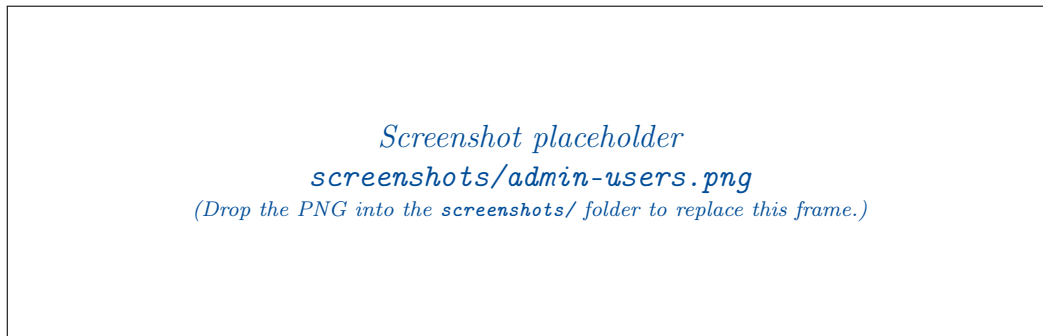


Figure 5.4: Users Management page: role filter, search, and the create-user modal driven by the user-management-service.

Staff Management Page

Description: Specialised CRUD for non-player staff (Sport Manager, Team Manager, Head Coach, Assistant Coach, Doctor, Physio, Fitness Coach, Scout, Sponsor, Performance Analyst). Same back-end as Users Management, but the modal pre-filters role choices to staff roles and exposes staff-specific fields (specialty, licence number, team assignment).

Key Features:

- Role chip column that colour-codes each staff member by functional area.
- Team assignment dropdown sourced live from `player-management-service` / `teams`.
- Bulk activate / deactivate for end-of-season offboarding.

Result: Streamlined onboarding of every non-player actor in the club, with role assignment and team binding in a single workflow.

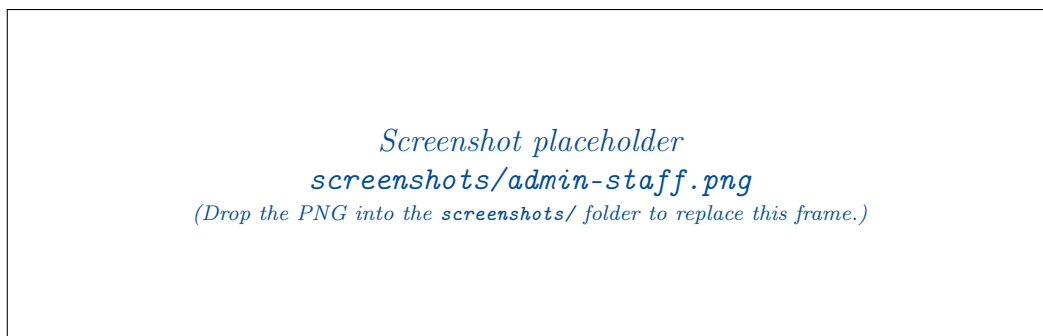


Figure 5.5: Staff Management page with the role-coloured chip column.

Notifications Page (Admin View)

Description: Broadcast surface backed by `notification-mail-service`. Admins can issue notifications to any subset of roles or to a specific user.

Key Features:

- Targeted dispatch by role group, by individual user, or system-wide.
- Priority levels (low / normal / high / critical) reflected in the UI styling for recipients.
- Delivery channels (in-app, email) chosen per broadcast.
- History list of past broadcasts with delivery counts.

Result: A reliable broadcast plane for governance announcements without requiring email-server access.

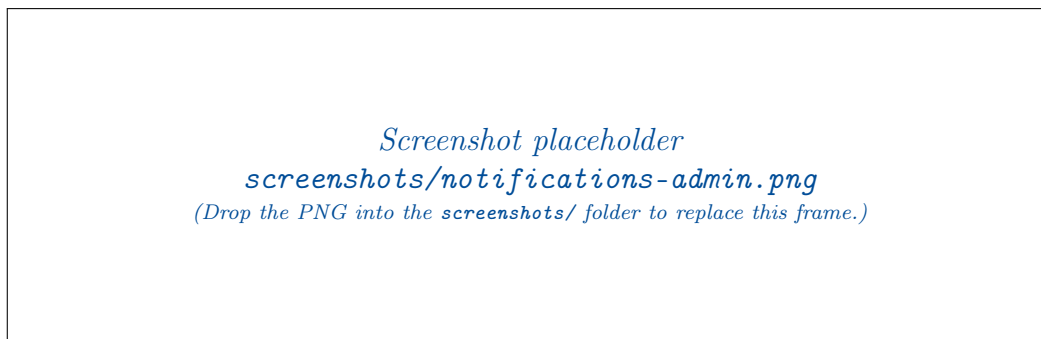


Figure 5.6: Admin broadcast view with priority chips and per-role targeting.

System-wide Read Access

The admin role passes the gateway's authorisation check for every read endpoint — players, teams, matches, training, medical reports, analytics, sponsor contracts — so admins can audit any data in the system from the same UI used by domain specialists.

Admin: Backend Wiring

- **Endpoints used:** GET/POST/PATCH/DELETE `/users`, GET/POST `/staff`, POST `/notifications/broadcast`
- **Owning services:** `user-management-service` for identity / profile CRUD, `notification-mail-service` for broadcasts.
- **Tables touched:** `mscms_user.user_profile`, `mscms_user.staff_profile` (and the role-specific profile sub-tables for doctors, physios, scouts, sponsors), `mscms_notification.notifications`, `mscms_notification.alerts`.
- **External calls:** Keycloak Admin API (create user, assign realm role, deactivate / delete).
- **Gateway guard:** every endpoint above is annotated with `@PreAuthorize("hasRole('ADMIN')")` on the server side, in addition to the front-end `permissions.js` check.

Admin: User Journey

1. Open the sidebar and click *User Management*.
2. Filter by role or search by username to find existing users, or click *+ Add User* to start a fresh create flow.
3. In the modal, pick the target role from the dropdown (which dynamically reveals role-specific fields such as *specialty* or *team assignment*).
4. Submit the form. The toast confirms creation; the list refreshes within 300 ms.
5. For sensitive changes (role demotion, deactivation), an inline confirmation modal prevents accidental clicks.
6. Broadcasts are issued from the bell icon's *Compose* button — pick priority, target audience and channel.

Admin: Validation & Error Handling

- **Client-side:** username regex `/^[a-zA-Z0-9._-]+$` matching Keycloak's own rules so the create call cannot fail with `error-username-invalid-character`.
- **Server-side:** duplicate username / email return 409 with a structured body the UI maps to a precise inline error.
- **Keycloak rollback:** if the profile insert in `mscms_user` fails after Keycloak provisioning has succeeded, the service issues a compensating `DELETE` against the new Keycloak user so the two sides cannot drift.
- **Audit:** every create / role-change / delete writes a row into `mscms_notification.alerts` marked `AUDIT` so any later review can reconstruct the sequence of admin actions.

5.5 Sport Manager Role Implementation

The Sport Manager owns club-wide strategic decisions across every discipline.

Sport Manager Dashboard

Description: Cross-discipline KPI surface that aggregates data from `reports-analytics-service` and rolls up team-level analytics across football, basketball, handball and volleyball.

Key Features:

- Per-discipline win/draw/loss tally.
- Pending sponsor offers count with one-click navigation to the Sponsorship page.
- Transfers awaiting approval (incoming, outgoing, call-ups).
- Scout-report inbox preview.

Result: The sport manager sees every cross-cutting decision queued up for their attention in one place.



Figure 5.7: Sport Manager dashboard with per-discipline KPI cards and the pending sponsor/transfer queue.

Transfers Page

Description: Manages the three transfer types modelled in `player-management-service`: *incoming* (a player joining from outside the club), *outgoing* (a player leaving), and *call-ups* (a player loaned to a national team).

Key Features:

- Tabbed view: Incoming / Outgoing / Call-ups.
- Status badges: PENDING, ACCEPTED, REJECTED, COMPLETED.
- Approve / reject actions that emit a Kafka event picked up by the notification service.
- Link to the underlying scout report (for incoming) or national-team call-up record.

Result: A complete audit trail for every player movement, linked back to the scouting and notification fabrics.

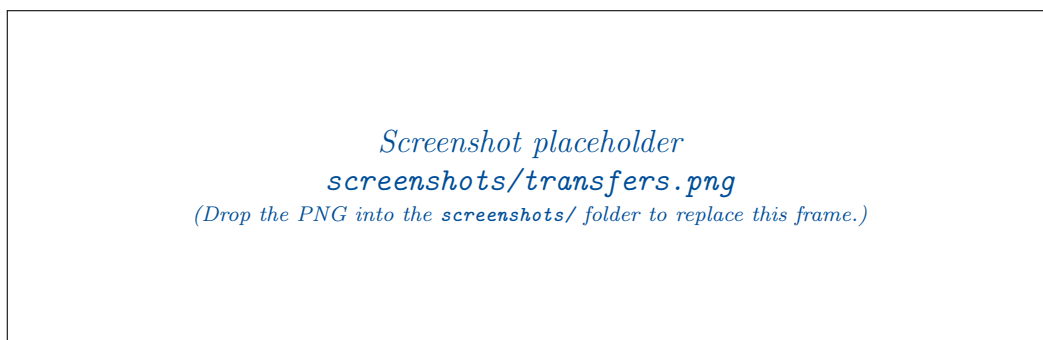


Figure 5.8: Transfers page showing the Incoming / Outgoing / Call-up tabs and status pipeline.

Sponsors Page

Description: CRUD over the `sponsor_contract_offers` table in `mscms_reports`. Sponsor offers are created by users with the *Sponsor* role and queued for sport-manager approval.

Key Features:

- Status pipeline view: PENDING, ACCEPTED, REJECTED.
- Offer detail panel showing value, duration, conditions and the originating sponsor company.
- Accept / reject buttons that update the database and emit a notification to the originating sponsor.

Result: The Spotify and Nike offers in the demo seed land **ACCEPTED**, while Damm and Beko sit **PENDING** — demonstrating the full workflow end-to-end with deterministic data.

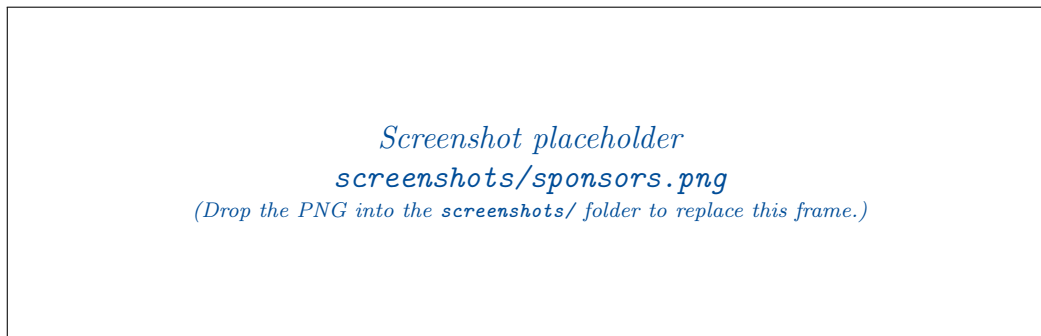


Figure 5.9: Sponsors page with the Spotify / Nike / Damm / Beko offers in the demo seed data.

Cross-discipline Player and Team Views

The Sport Manager has read access to every team and player across disciplines, including detailed analytics roll-ups not available to team managers.

Sport Manager: Backend Wiring

- **Endpoints used:** GET /transfers/in, GET /transfers/out, GET /player-callups, PATCH /transfers/in/{id}, PATCH /transfers/out/{id}, PATCH /sponsors/{id}, GET /team-analytics, GET /scout-reports.
- **Owning services:** player-management-service (transfers, call-ups), reports-analytics-service (sponsor offers, scout reports, team analytics).
- **Tables touched:** player_transfer_incoming, player_transfer_outgoing, players-call-up-re, sponsor_contract_offers, scout_reports, team_analytics.
- **Kafka events emitted:** TransferDecided, SponsorDecided, consumed by the notification service to alert scouts, sponsors and team managers.

Sport Manager: User Journey

1. Land on the cross-discipline dashboard; the four headline cards (W/D/L per discipline, pending sponsor offers, transfers awaiting approval, scout-report inbox) reveal what needs attention today.

2. Drill into Transfers to review the pipeline; each incoming request is rendered alongside the scout report that originated it.
3. Accept or reject with a single click; the chip flips to **ACCEPTED** / **REJECTED** immediately while the notification fans out in the background.
4. Switch to Sponsorship to clear pending sponsor offers using the same accept/reject pipeline.
5. Open the Team Analytics page (also under Analytics Overview) for a season-to-date view across football, basketball, handball and volleyball.

Sport Manager: Validation & Error Handling

- Transfer / sponsor PATCH calls reject any status transition that is not **PENDING** → **ACCEPTED** or **PENDING** → **REJECTED** (returns 409).
- Accepting an incoming transfer triggers a server-side transaction across three tables; partial failure rolls back the whole accept so we never have a transfer marked accepted without a roster + contract row.
- The UI surfaces 409 conflicts as inline error chips on the affected row, not as global toasts, so the manager keeps their place in the list.

5.6 Team Manager Role Implementation

The Team Manager owns one specific team within one discipline and is the operational counterpart to the Head Coach.

Team Manager Dashboard

Description: Single-team operational view scoped to the team the manager is assigned to.

Key Features:

- Roster snapshot with active / injured counts.
- Upcoming fixtures with status badges.
- Pending training plans awaiting approval.
- Recent match events.

Result: A focused, single-team cockpit without the cross-discipline noise of the Sport Manager view.



Figure 5.10: Team Manager dashboard scoped to a single team.

Players Page

Description: Single-team roster CRUD with positions filtered to the discipline (e.g. handball positions are HB_LEFT_WING, HB_RIGHT_WING, etc.).

Key Features:

- Add Player modal posts to POST /players with a 17-field schema that provisions both a Keycloak user and a player_profile row in one call.
- Position dropdown bound to the discipline (football positions for FC Barcelona, basketball positions for Bàsquet, handball positions for Handbol, volleyball positions for CV).
- Edit player with form pre-filled from GET /players/{id}.
- Contract status and rehabilitation banners surface medical flags inline.

Result: A discipline-aware roster surface where adding a player creates both an identity and a profile in one round-trip, preventing the orphan-record drift typical of Keycloak-backed apps.



Figure 5.11: Players page with the discipline filter chips and the Add Player modal.

Teams Page

Description: List of every team across disciplines. Team Managers see read-only entries for teams other than their own, while Sport Managers and Admins can edit any team.

Key Features:

- Discipline filter chips with the MdOutlineSportsVolleyball icon for the Teams navigation entry.
- Crest, founding year and coach summary per team.
- Drill-down to roster, schedule and historical analytics.

Result: A consistent multi-discipline directory that respects team-manager scope.



Figure 5.12: Teams page covering FC Barcelona, Bàsquet, Tennis Academy, CV and Handbol.

Matches Page

Description: Lists matches owned by the team. Team Managers can create new fixtures, attach formations, publish line-ups and log post-match reviews.

Key Features:

- Status pipeline: SCHEDULED, LIVE, FINISHED.
- Match detail panel showing opponent, venue, kick-off, formation and assigned line-up.
- Event log for goals, cards, substitutions (head coach edits, team manager reads).
- Linked post-match analysis from the Performance Analyst.

Result: Every match from the demo seed (FC Barcelona vs. Real Madrid **FINISHED**, vs. Atlético **SCHEDULED**, basketball vs. RM Baloncesto) is visible end-to-end with formations and events.



Figure 5.13: Matches page with the status pipeline and the per-match detail panel.

Team Manager: Backend Wiring

- **Endpoints used:** GET/POST/PATCH/DELETE /matches, GET/POST /match-formations, GET/POST /match-lineups, GET/POST /match-events, GET/POST /match-performance-reviews, GET /players?teamId=.
- **Owning service:** training-match-service.
- **Tables touched:** matches, match_formation, match_lineups, match_events, match_performance.
- **Cross-service reads:** player-management-service / teams (for the team dropdown), reports-analytics-service / match-analyses (for the post-match analysis panel).

Team Manager: User Journey

1. Open the team dashboard, scoped automatically to the team the manager is assigned to via their `staff_profile.teamId`.
2. Click *New Match* to schedule an upcoming fixture; the modal validates opponent, venue and kick-off time client-side.
3. Open the saved match, attach a formation from the formations library (or create one), then drag-and-drop players to fill each slot under *Lineup*.
4. During the match, log goals, cards and substitutions through the events panel; each entry hits POST /match-events with the `minute + eventType + playerKeycloakId` payload.
5. After the match, request a Performance Review and link the analyst's Match Analysis from the Reports page.

Team Manager: Validation & Error Handling

- **Schedule conflict guard:** the matches POST rejects a kickoff that overlaps another scheduled match for the same team (returns 409).
- **Discipline-aware position picker:** the line-up editor filters available positions by the match's `sportType`, so a basketball line-up never offers `HB_LEFT_WING`.
- **Player-availability guard:** players currently in `ACTIVE` injury status are surfaced with a red badge in the slot picker so the team manager doesn't accidentally select an unavailable player.

5.7 Head Coach & Assistant Coaches Role Implementation

Coaching staff own training and tactical decisions. The Head Coach has full edit rights, Assistant Coaches have curated edit rights on training plans and Goalkeeper Coaches are scoped to GK-position players.

Coach Dashboard

Description: Coach-centric pulse view: today's sessions, players currently in rehab (read-only feed from the medical service), next fixture, and outstanding training assessments.

Key Features:

- Today's sessions strip pulled from `training-match-service / sessions`.
- Rehab read-only sidebar pulled from `medical-fitness-service / injuries?status=ACTIVE`.
- Quick-create training session and quick-attach drill buttons.

Result: The coach starts every day with a single screen covering attendance, fitness and tactical state.



Figure 5.14: Coach Dashboard with today's sessions and the rehab read-only sidebar.

Training Page

Description: Full lifecycle of training sessions: plans, sessions, drills, attendance and per-player assessments.

Key Features:

- Create training plan with weekly cadence.
- Attach drills from the drill library (4 demo drills seeded).
- Record attendance per session per player.
- Write per-player assessments with rating and free-text observations.

Result: Every demo session, plan, drill, attendance row and assessment is consistent with the seeded match calendar, demonstrating the end-to-end coaching loop.

Formations & Line-ups

Description: Tactical authoring tool stored in `training-match-service` as a `formation` entity with attached `lineup` rows referencing players by `playerId`.

Key Features:

- Drag-and-drop player slots on a tactical board.
- Position-aware constraints (a goalkeeper slot only accepts a GK-position player).



Figure 5.15: Training page: plans, sessions, drills, attendance and assessments in a single view.

- Save as draft / publish for the upcoming match.
- Versioned formations reusable across matches.

Result: The 1 formation + 4 line-ups in the demo seed map cleanly onto the displayed tactical board.

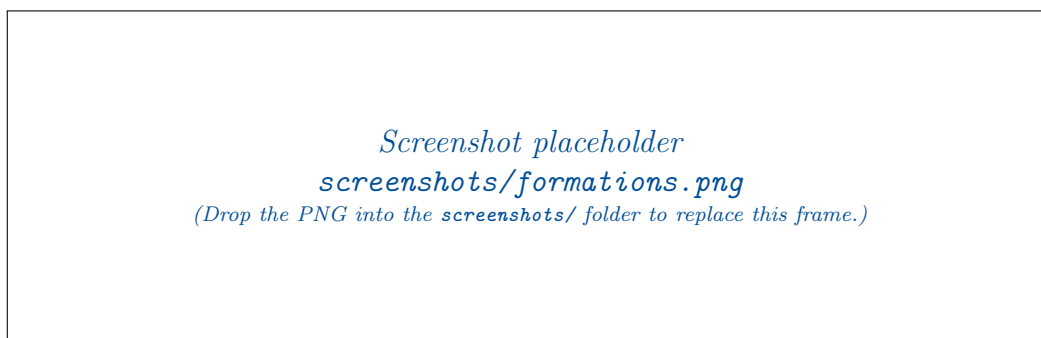


Figure 5.16: Formations & Line-ups editor with the tactical board.

Match Events Log

Description: In-game event recorder for goals, cards, substitutions and other key actions. Backed by the `match_events` table with a `minute` and `event_type` per row.

Key Features:

- Live-mode minute-by-minute entry.
- Event-type chips (GOAL, YELLOW, RED, SUB_IN, SUB_OUT).
- Per-event player attribution.
- Auto-rollup into the match's final stat-line.

Result: The 4 demo events drive the visible match recap and the per-player stats panel.

5.8 Medical Staff Role Implementation

Medical staff own everything in `mscms_medical`: injuries, diagnoses, treatments, rehabilitations, recovery programs, fitness tests and training loads.

Medical Dashboard

Description: A condensed view of the medical inbox tuned to the staff member's specialty.

Key Features:

- Active injuries count grouped by severity.
- Players currently in rehab.
- Fitness tests scheduled for the next 14 days.
- Training-load alerts for players whose load exceeds the configured chronic-load ratio.

Result: A medical staff member sees the day's priorities at a glance.



Figure 5.17: Medical dashboard with active injuries grouped by severity.

Injuries Page

Description: CRUD over the `injuries` table with linked diagnoses, treatments and rehabilitation programs.

Key Features:

- Open new injury with player picker, injured-area chip and severity badge.
- Attach diagnosis records authored by the doctor.
- Attach treatment records authored by the physio.
- Attach rehab program authored by the fitness coach.
- Status pipeline: `ACTIVE`, `IN_REHAB`, `RECOVERED`.

Result: The 2 demo injuries map to 2 diagnoses, 2 treatments, 2 rehabs and 2 recovery programs, demonstrating the full cross-document chain.

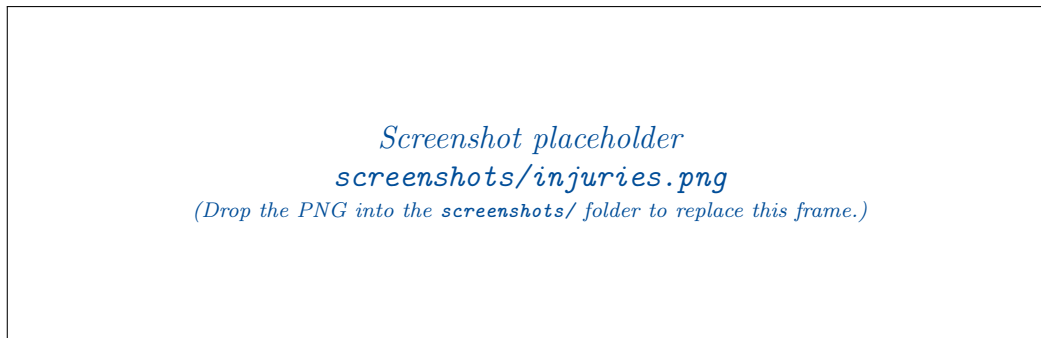


Figure 5.18: Injuries page with the diagnosis / treatment / rehab side panel.

Fitness Tests & Training Load

Description: Fitness Coaches author `fitness_tests` rows (5 demo entries) and `training_loads` rows (5 demo entries) per player per session.

Key Features:

- Test-result entry with metric per discipline (VO₂ max for endurance, vertical jump for power).
- Training-load trend chart per player.
- Cross-reference against coach-authored training sessions.

Result: A complete fitness/load history that informs both the coaching staff (load management) and the medical staff (injury-risk profiling).

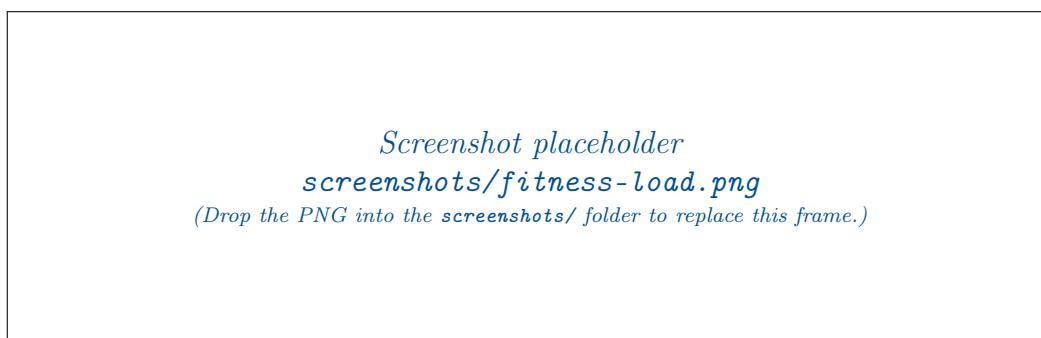


Figure 5.19: Fitness tests and training-load trend chart per player.

Medical Staff: Backend Wiring

- **Endpoints used:** GET/POST/PATCH/DELETE /injuries, GET/POST /diagnoses, GET/POST /treatments, GET/POST /rehabilitations, GET/POST /recovery-programs, GET/POST /fitness-tests, GET/POST /training-loads.
- **Owning service:** medical-fitness-service.
- **Tables touched:** injuries, diagnoses, treatments, rehabilitations, recovery_programs, fitness_tests, training_loads.
- **Kafka events emitted:** InjuryReported, RehabUpdated, FitnessTestRecorded.

Medical Staff: User Journey

1. Open the medical dashboard — active injuries grouped by severity, players currently in rehab, and any chronic-load alerts.
2. For a new injury: pick the player, the body part, severity and add a description. The new `injury` row defaults to `ACTIVE` and immediately appears on the head coach's availability strip.
3. As diagnosis details emerge, attach `diagnosis` and `treatment` records; each is linked back to the parent injury for traceability.
4. Open a rehab record, advance the stage, log progress notes, and (when complete) flip the status to `COMPLETED` — the parent injury cascades to `RECOVERED` automatically.
5. Fitness coaches log `fitness_tests` and `training_loads` from the same nav, with the chart view auto-updating from the latest entry.

Medical Staff: Validation & Error Handling

- Diagnoses, treatments and rehab records cannot be created without a parent injury (`injuryId` is mandatory and validated FK-side).
- Status transitions on injuries (`ACTIVE` → `IN_REHAB` → `RECOVERED`) are enforced server-side; the UI hides the buttons that would request an invalid transition.
- Training-load values outside the 0.0 - 1.0 intensity range return 400 with a precise field error.
- Fitness coaches and doctors share the same dashboard but gateway-level role checks ensure each can only write into the tables their realm role covers (e.g. a physio cannot author a `diagnosis` record).

5.9 Scout Role Implementation

The Scout role discovers and evaluates external talent.

Scout Dashboard

Description: Pipeline view of in-progress scouting work.

Key Features:

- Reports in draft.
- Reports submitted and awaiting sport-manager decision.
- Recently approved transfers originating from this scout.

Result: A scout sees the conversion rate of their own pipeline.

Outer Players Page

Description: Catalogue of external players (rival teams and free agents) modelled in `player-management-service`. 4 demo outer players are seeded (Real Madrid CF, Atlético, Sevilla, RM Baloncesto representatives).

Key Features:

- Filter by team, position, age range.
- Per-player profile with cross-references to existing scout reports.
- Quick `Create scout report` action.

Result: A single catalogue of every prospect the club is watching, with workflow continuity to the scout-report tool.



Figure 5.20: Outer Players catalogue with rival-team filtering.

Scout Reports Page

Description: CRUD over `scout_reports` rows with structured ratings (technical, physical, tactical, mental, potential) and free-text observations.

Key Features:

- Five-axis radar chart for technical / physical / tactical / mental / potential scores.
- Recommendation chip (`SIGN`, `MONITOR`, `REJECT`).
- Forwards selected reports to the sport manager.

Result: The 4 demo scout reports drive the incoming-transfer queue visible to the sport manager.

Scout: Backend Wiring

- **Endpoints used:** `GET/POST /outer-players`, `GET/POST /outer-teams`, `GET/POST/PATCH/DELETE /scout-reports`, `POST /transfers/in` (forward a scouted prospect into the sport manager's queue).
- **Owning services:** `player-management-service` (outer players, outer teams), `reports-analytics-` (scout reports).



Figure 5.21: Scout Reports page with the five-axis radar chart.

- **Tables touched:** `outer_players`, `outer_teams`, `scout_reports`, `player_transfer_incoming`.
- **Kafka events emitted:** `ScoutReportSubmitted`, consumed by the notification service to land the report in the sport manager's inbox.

Scout: User Journey

1. Open the Outer Players catalogue, filter by rival team or position, and inspect prospects.
2. Click *New Report* on a player; the modal pre-fills the outer player ID and team relation.
3. Score the five axes (technical, physical, tactical, mental, potential), add free-text strengths/weaknesses, and pick a recommendation chip.
4. Submit. The report appears in the sport manager's transfers inbox; the scout's dashboard shows it under *Submitted / Awaiting decision*.
5. When the sport manager accepts the linked transfer, the scout sees the report move to *Approved* on their dashboard.

Scout: Validation & Error Handling

- All five rating axes must be in the 1-10 range or the POST returns a 400 with the offending field name.
- The recommendation chip is required — the modal disables the submit button until one of `SIGN`, `MONITOR`, `REJECT` is selected.
- Forwarding a report to the sport manager creates a `player_transfer_incoming` row pre-populated with the outer player + outer team relations, so the sport manager doesn't need to re-enter any of the player's identifying data.

5.10 Sponsor Role Implementation

The Sponsor role represents commercial partners.

Sponsor Dashboard

Description: Sponsor-scoped surface showing their active contract, pending offers and recent club news the sponsor would care about (matches, transfers, fan reach).

Sponsorship Page

Description: CRUD over their own `sponsor_contract_offers` rows.

Key Features:

- Submit new offer: value, duration, conditions, sport targeting.
- Track offer status (PENDING, ACCEPTED, REJECTED).
- Notifications on decision.

Result: The Spotify and Nike sponsors in the seed appear with **ACCEPTED** contracts; Damm and Beko appear **PENDING** and surface to the sport manager's decision queue.



Figure 5.22: Sponsorship page from the sponsor's own perspective: own contracts and pending offers.

5.11 Performance Analyst Role Implementation

The Performance Analyst is the heaviest consumer of the `reports-analytics-service` and the ML services.

Analytics Page

Description: Overview surface that aggregates five back-end calls in parallel: team analytics, player analytics, match analytics, training analytics and active injuries. The result is denormalised in the front-end into a single canvas.

Key Features:

- Headline KPIs: wins, goals scored, goals conceded, injuries.
- Player leaderboards (top scorers, top assists, fewest minutes on the bench).
- Per-team form curves over the season.
- Training-load distribution histogram.

Result: The Analytics Overview becomes the single source of truth for board-level reporting, replacing what would otherwise be multiple spreadsheets and reports.



Figure 5.23: Analytics Overview aggregating five APIs into one canvas.

Match Analyses Page

Description: Structured per-match analysis authored by the Performance Analyst, linked to the underlying match.

Key Features:

- Pre-filled metrics from the match-event log.
- Free-text narrative section with embedded headings.
- Recommendations list consumed by the coaching staff.

Result: The 1 demo match analysis is visible to coaches, team managers and sport managers — a single document threaded through the workflow.



Figure 5.24: Match Analyses page with the narrative-plus-metrics layout.

AI & Predictions Page

Description: Front-end for the two FastAPI ML services: `ml-match-predictor` (port 9000) and `ml-player-rating` (port 9001). The front-end calls them through a Next.js rewrite proxy (`/ml-proxy/match/*` and `/ml-proxy/player/*`) to avoid CORS pre-flights.

Key Features:

- Match prediction form: home team, away team, referee selector with an *Other* fallback.
- Player rating form bounded to a curated list of 20 known players from the training dataset (avoids *Player not found* errors).
- Confidence / probability display.

- Loading and error states with toast feedback.

Result: Non-technical staff consume ML predictions through the same UI as any other domain feature, with no awareness of the Python services running underneath.

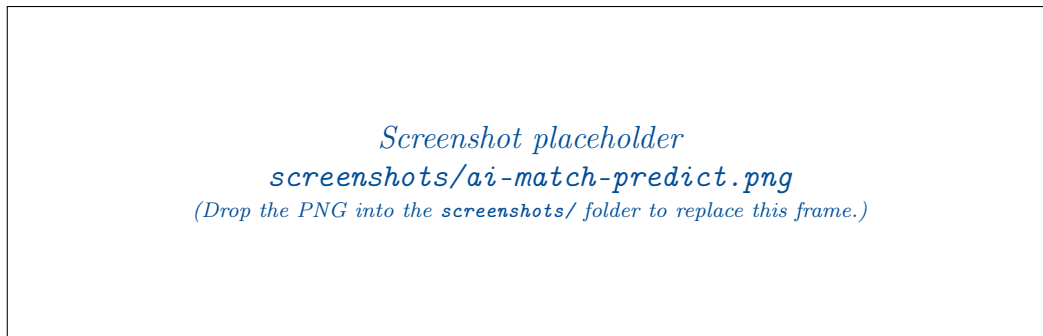


Figure 5.25: AI Match Predictor: home/away selectors, referee picker and the outcome probability bars.



Figure 5.26: AI Player Rating: known-players autocomplete and the predicted vs. actual rating.

Reports Page

Description: Browseable list of structured reports (match analyses, team analytics, training analytics) with download support. The `handleDownload` action generates a `.txt` blob client-side from the JSON payload.

Key Features:

- Filter by report type and date range.
- One-click download to `.txt`.
- Linked source records (match, team, training session).

Result: A self-service report library that does not require the analyst to copy data into Word / PDF for every consumer.



Figure 5.27: Reports page with download-to-.txt actions.

Performance Analyst: Backend Wiring

- **Endpoints used:** GET /team-analytics, GET /player-analytics, GET /training-analytics, GET /match-analyses, GET /injuries?status=ACTIVE, POST /match-analyses, GET /ml-proxy/match/predict, GET /ml-proxy/player/predict/{name}.
- **Owning services:** reports-analytics-service for all analytics CRUD; ml-match-predictor (port 9000) and ml-player-rating (port 9001) for AI inference.
- **Tables touched:** team_analytics, player_analytics, training_analytics, match_analysis.
- **Frontend rewrite:** Next.js next.config.mjs rewrites /ml-proxy/match/* → http://localhost:9000 and /ml-proxy/player/* → http://localhost:9001 so the browser sees same-origin calls.

Performance Analyst: User Journey

1. Open the Analytics Overview — five parallel API calls populate headline KPIs and player leaderboards in under a second.
2. Open a specific match from Match Analyses; pre-filled metrics come from the match-event log, leaving the analyst free to focus on narrative and recommendations.
3. Submit the analysis; coaches and team managers see it on their dashboards.
4. Switch to AI & Predictions to run a match outcome prediction (pick home / away / referee) or a player rating (pick from the curated dataset of known players).
5. Download structured reports from the Reports page as .txt blobs for distribution.

Performance Analyst: Validation & Error Handling

- Match analyses cannot be submitted without a `matchId` that references an existing match (the modal queries the Matches endpoint to populate the autocomplete).
- Average rating fields are clamped 0-10; out-of-range values surface as inline errors.
- The AI match prediction returns probabilities summing to 1.0 within a small floating-point tolerance; the front-end rejects (and toasts) any response that violates this.
- The AI player rating dropdown lists only the twenty players known to the trained model, eliminating *Player not found* 404s entirely.

5.12 Player Role Implementation

Players are first-class users with read-only access to their own performance, schedule and medical state.

Player Dashboard

Description: Personal cockpit: today's session, next match, last assessment, current medical status.

Key Features:

- Today's training session card with attendance state.
- Next match card with countdown.
- Latest assessment summary (coach-authored).
- Active injury / rehab status (medical-authored, read-only).

Result: A player sees the operational state of their own career on one screen, without needing to ask staff.

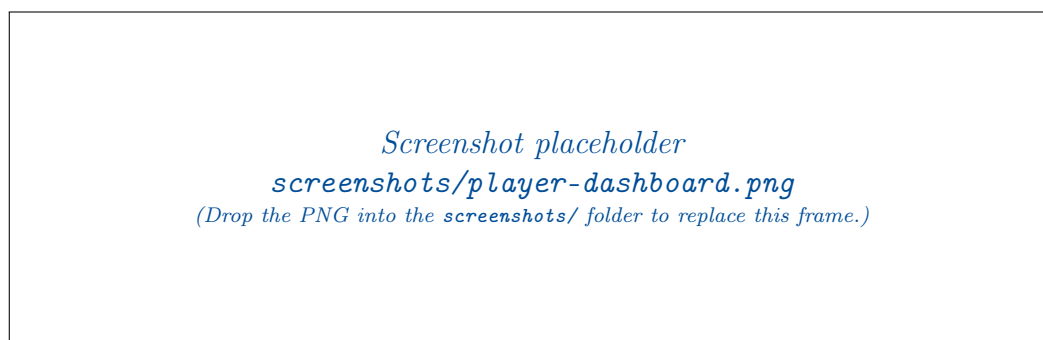


Figure 5.28: Player Dashboard: today's session, next match, latest assessment and active medical status.

Personal Stats Page

Description: Aggregated per-player stats sourced from the `match_events` and `player_match_statistics` tables.

Key Features:

- Goals, assists, minutes per match.
- Training attendance percentage.
- Trend line over the season.

Result: Transparent, real-time access to their own performance — empowering players to negotiate, self-assess and stay motivated.

Personal Medical View

Description: Read-only view of own injury, diagnosis, treatment and rehab records.

Result: Player can follow their own recovery journey without asking medical staff for daily updates.

5.13 Fan Role Implementation

Fans have a curated read-only surface: Matches, Players, Teams, Training (public-facing parts only) and Reports (public parts only). Medical, scouting, sponsorship and finance views are completely hidden from the navigation, and all *Add / Edit / Delete* buttons are removed across pages where the Fan still has read access.

Fan Dashboard

Description: Public-facing club hub.

Key Features:

- Upcoming fixtures across all disciplines.
- Latest match result.
- Player of the week (sourced from analytics).
- Club news strip.

Result: Fans get an integrated club experience inside the same platform staff use — a feature competitors like Hudl and Catapult do not provide.

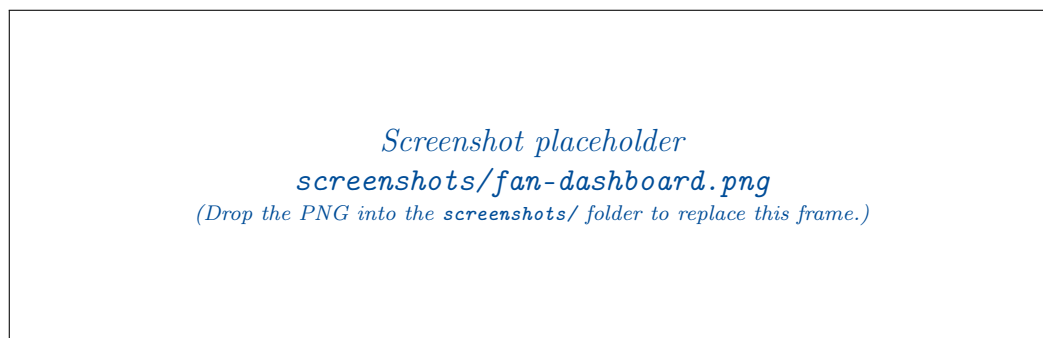


Figure 5.29: Fan Dashboard: read-only club hub.

Fan Matches / Players / Teams / Training / Reports Pages

Each of these pages reuses the same React component as the corresponding staff page, but with conditional rendering driven by the `canAccess` helper from `permissions.js`: *Add / Edit / Delete* buttons are hidden, edit-only fields are collapsed and the breadcrumbs do not expose internal-only entities (scout reports, sponsor offers, injuries).

Result: A polished public surface that shares its components with the staff surface — new fields added to staff views appear automatically (read-only) to fans, with no double maintenance.



Figure 5.30: The same Matches component rendered for a fan: Add / Edit / Delete actions are suppressed by `canAccess`.

5.14 Notifications (All Roles)

Description: Bell icon in the navbar opens a panel powered by `notification-mail-service`. Notifications are produced by back-end Kafka consumers on key events (transfer decision, injury opened, sponsor decision, line-up published) and fanned out to the relevant role group.

Key Features:

- In-app feed with unread counter.
- Priority styling (low / normal / high / critical).
- Optional email delivery for high / critical notifications.
- Per-user read state persisted by the service.

Result: Every staff role and every player receives the right information at the right moment without the noise of a club-wide mailing list.



Figure 5.31: Navbar notifications panel with unread counter and priority styling.

5.15 Settings (All Roles)

Description: The settings page presents a tabbed view of profile, security and preferences. `ProfileTab` reads `name`, `email` and `role` from the decoded JWT, guaranteeing that what the user sees is exactly what Keycloak believes about them.

Key Features:

- Profile view sourced from the JWT plus profile-row enrichment.
- Password change link redirecting to Keycloak account console.
- Language and theme preferences stored client-side.

Result: A consistent identity surface across all 19 roles, with no drift between the JWT and the visible profile.

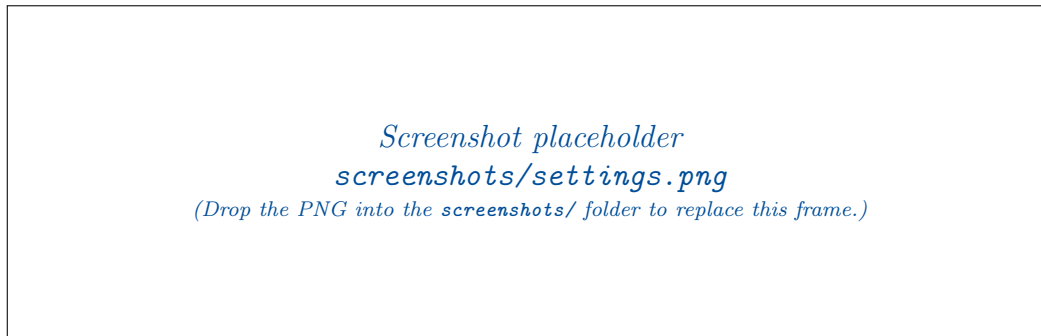


Figure 5.32: Settings page: Profile tab driven by the decoded JWT.

5.16 Cross-cutting Implementation Notes

Permissions source of truth: `sports-club-main/my-app/src/lib/permissions.js` contains `SIDEBAR_SECTIONS` (the navigation map) and `canAccess` (the per-route role check). Both the `Sidebar` component and the Next.js middleware consume this file — there is no second implementation of the rules.

Idempotent seeders: Every microservice has a `*DataSeeder.java` bootstrap class guarded by a `count() > 0` check. Restarting the stack does not duplicate data. The notification service got a brand-new `NotificationDataSeeder` created from scratch for this project; the others were extended to seed FC Barcelona’s real squads, contracts, transfers, sponsor offers and analytics.

Single docker-compose: `MSCMS-main/docker-compose.full.yml` brings the entire stack up. `build:` directives ensure local source is rebuilt; ML services are included on ports 9000 and 9001.

Front-end ML proxy: `sports-club-main/my-app/next.config.mjs` rewrites `/ml-proxy/match/` to `http://localhost:9000` and `/ml-proxy/player/*` to `http://localhost:9001`, avoiding CORS pre-flights when calling Python from the browser.

Result of Implementation: The system is end-to-end functional with deterministic FC Barcelona seed data, 19 working roles, full RBAC at gateway and middleware, embedded ML predictions and a polished UX — delivering on every goal listed in Section 1.5.

Chapter 6

Conclusion and Future Work

Multi-Sport Club Management System

MSCMS is an integrated platform designed to overcome the limitations of existing sports management tools, offering a unified solution for multi-discipline club operations and analytics. Key features include:

- **Nineteen User Roles:** Tailored dashboards across leadership, coaching, medical, scouting, commercial and end-user actors.
- **Multi-Sport Modelling:** Native support for football, basketball, handball and volleyball under a single data model.
- **Microservice Architecture:** Seven Spring Boot services + two FastAPI services with Eureka discovery and a single API gateway.
- **Embedded ML Predictions:** Match outcome and player rating served behind a same-origin proxy.
- **Centralised Analytics:** One Overview page aggregating five APIs.
- **Fan-facing Surface:** Read-only public hub integrated in the same platform staff use.
- **Event-driven Notifications:** Kafka- and RabbitMQ-backed role-aware fan-out.

6.1 Comparison Between Sports Management Platforms

Feature / Tool	Hudl	Catapult	SAP Sports One	TeamSnap	MSCMS
19-role RBAC (sport mgr, doctor, scout, sponsor, fan, ...)	Limited	Limited	Partial	Limited	Yes

Multi-sport modelling (football + basketball + handball + volleyball)	Limited	Limited	Partial	Limited	Yes
Microservice architecture (Eureka + gateway + Kafka)	No	No	Partial	No	Yes
Embedded ML match prediction	No	No	Partial	No	Yes
Embedded ML player rating	No	Partial	Partial	No	Yes
Medical & fitness module (injuries, rehab, fitness tests)	No	Partial	Yes	No	Yes
Scout reports + outer players catalogue	No	No	Yes	No	Yes
Sponsor offers workflow	No	No	Limited	No	Yes
Transfer / call-up workflow	No	No	Partial	No	Yes
Fan-facing surface in same platform	No	No	No	Limited	Yes
Self-hostable (Docker Compose)	No	No	No	No	Yes
Open architecture (Spring Boot + Next.js + FastAPI)	No	No	No	No	Yes

6.2 Conclusion

The analysis of Hudl, Catapult, SAP Sports One, TeamSnap and Kitman Labs revealed critical gaps in role coverage, multi-sport modelling, embedded ML and integrated fan-facing surfaces. MSCMS surpasses these platforms by offering a single coherent system that combines nineteen roles, four disciplines, seven microservices and two ML services behind a single API gateway and a polished Next.js UI. By addressing the fragmentation and analytical blindness identified in Section 1.4, MSCMS delivers an innovative solution that streamlines club operations and fosters cross-domain decision-making.

6.3 What was delivered

This project delivered a working multi-sport club management platform that:

- Decomposes the back-end into seven Java microservices, two Python ML services and supporting infrastructure containers, all orchestrated by a single Docker Compose file.
- Authenticates every request against a Keycloak realm with 19 roles, gating both the front-end navigation and the back-end endpoints with consistent role lists.

- Persists data across six PostgreSQL databases using the database-per-service pattern, with deterministic seed data that models FC Barcelona’s real organisational structure.
- Exposes a polished Next.js front-end with role-aware navigation, read-only fan mode, aggregated analytics, downloadable reports, AI predictions, and the full CRUD surface of every operational domain.

6.4 Lessons learned

The most expensive lesson was the cost of *contract drift* between the front-end, the back-end DTOs and the database schema. We resolved it by treating one file (`permissions.js` on the front-end and `SeedIds.java` on the back-end) as the source of truth for each cross-cutting concern. A second lesson was the importance of *idempotent seeders*: every service guards its seed code with a `count() > 0` check, which makes the system safe to redeploy without wiping volumes.

6.5 Future work

Several extensions would naturally fit on top of the existing architecture. The list below is split into *product extensions* (new user-facing surfaces) and *platform extensions* (operational upgrades to the existing stack).

Product Extensions

- **E-commerce Store.** A merchandise + ticketing surface wired into the same Keycloak identity and the existing fan navigation. The shop would sell branded kits, training gear, season tickets and match-day tickets, with Stripe (or local equivalents such as Paymob and Fawry) as the payment gateway and a new `ecommerce-service` owning a `mscms_shop` database (`products`, `orders`, `order_items`, `payments`). Revenue events would feed back into the `reports-analytics-service` so the sport manager’s dashboard can correlate merchandise revenue with on-pitch performance. Sponsor contracts could include co-branded product placements, surfacing sponsor-specific items inside the same store.
- **Mobile App.** A native iOS / Android client (React Native or Flutter) targeting players, coaches and fans. Players would receive push notifications for line-up announcements, training schedules and injury updates; coaches would log attendance and rate sessions in the field without needing a laptop; fans would get match alerts and ticket purchases from the e-commerce store above. The mobile app reuses the existing gateway and Keycloak realm with no new identity surface, and adds an `mscms-mobile` public client to the realm with PKCE authentication. Offline-first storage (Room / SQLite) caches the most recent squad, fixtures, training schedule and notification feed so the app remains usable on patchy connections at stadiums and training grounds.
- **Real-time match feed** via Server-Sent Events from `training-match-service`, surfaced as live tickers in the front-end and as push notifications in the mobile app.

- **Fan engagement layer.** Polls, player-of-the-match voting, predict-the-score mini-games and a points-and-badges system, all tied to the same Keycloak fan identity. This layer would consume Kafka events already published by the training-match-service rather than introducing new write paths into the operational schema.
- **Multilingual UI.** Localise the front-end strings into Catalan, Spanish, English and Arabic to match Barcelona's international fan base. The translation infrastructure (i18n JSON bundles) would also unlock the e-commerce store and mobile app for regional markets.

Platform Extensions

- **Centralised observability** via Prometheus + Grafana dashboards reading from each service's `actuator/prometheus` endpoint, with alerts wired into the existing notification-mail-service so on-call rotations receive incidents through the same channel staff already trust.
- **Distributed tracing** with OpenTelemetry / Jaeger to make cross-service request paths visible (especially useful for debugging the gateway → Kafka → notification fan-out).
- **Production ML pipeline** that retraines the prediction models on fresh match data, replacing the static joblib snapshots with a versioned model registry (MLflow). Model drift alerts would flow through the same notification fabric.
- **Kubernetes deployment** with horizontal pod autoscaling per service, replacing the single-host Docker Compose stack with a multi-node cluster that can absorb fan-traffic bursts on match days.
- **Granular audit log** capturing every state-changing admin action (user creation, role change, contract approval) for governance and compliance review.